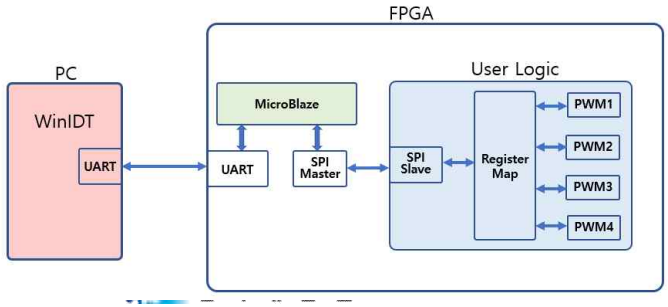
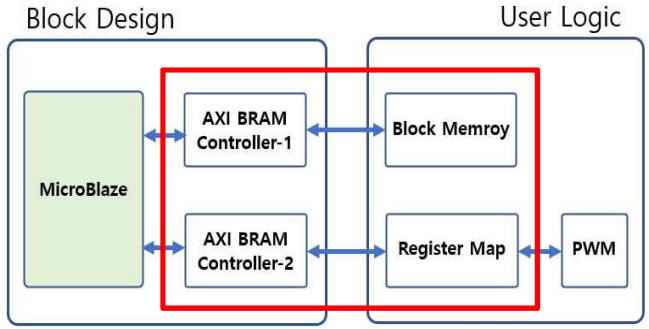
SoC 를 위한 *Peripheral* 설계

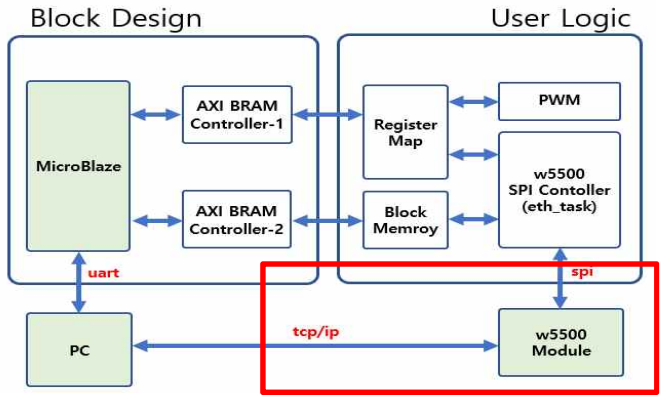
Reference : *MicroBlaze.v15 [IHIL]*

2025-06-24

Table of Contents







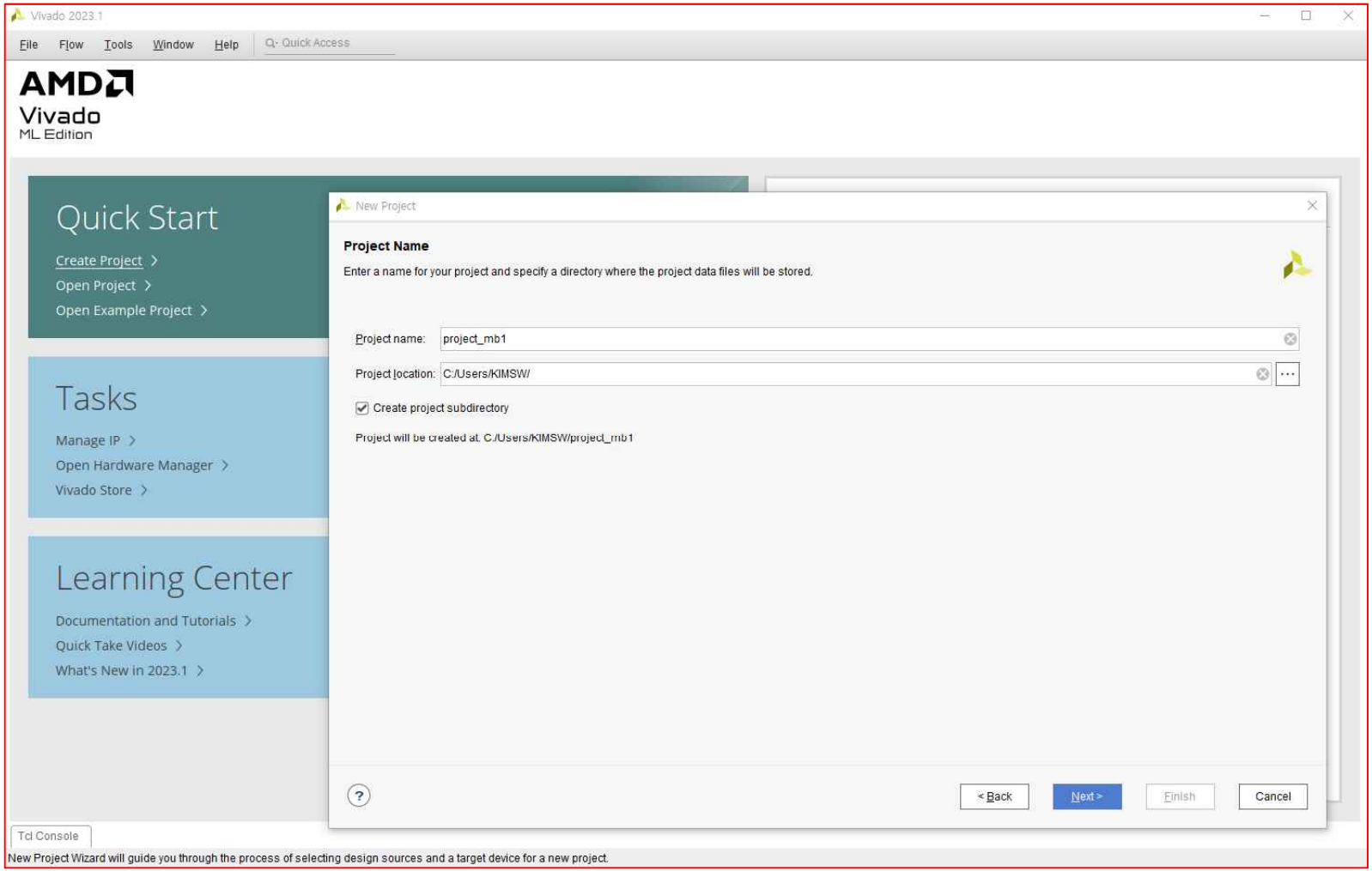
Hello World Implementation

- ***MicroBlaze***를 이용하여 ***Hello World*** 출력
  - ***MicroBlaze***는 ***Processor Core***와 ***Peripheral*** 이 분리
  - 원하는 Peripheral 추가 및 개수도 필요한 만큼 추가하여 사용가능
  - ***MicroBlaze Processor Core***와 ***Peripheral*** 중에 ***UART*** 를 추가하여 ***UART*** 포트를 통하여 ***Hello World*** 를 출력 기능 구현

- ***FPGA***에서 ***MicroBlaze***를 사용하는 목적
  - 사용자가 만든 ***User Logic***을 제어하고, ***Logic*** 으로 구현할 수 없는 부분을 ***MicroBlaze*** 로 구현
  - ***FPGA***에서 ***IP와 User Logic 을 생성해서 MicroBlaze*****사용하는 궁극적인 목적**은 ***HW*****로 구현하는 부분**과 ***SW*****로 구현하는 부분**을 나누어서 구현하고 **서로 간에 인터페이스를 구현**하여 외부에 추가적인 부품을 사용하지 않는 ***SoC (System on Chip)***를 구현

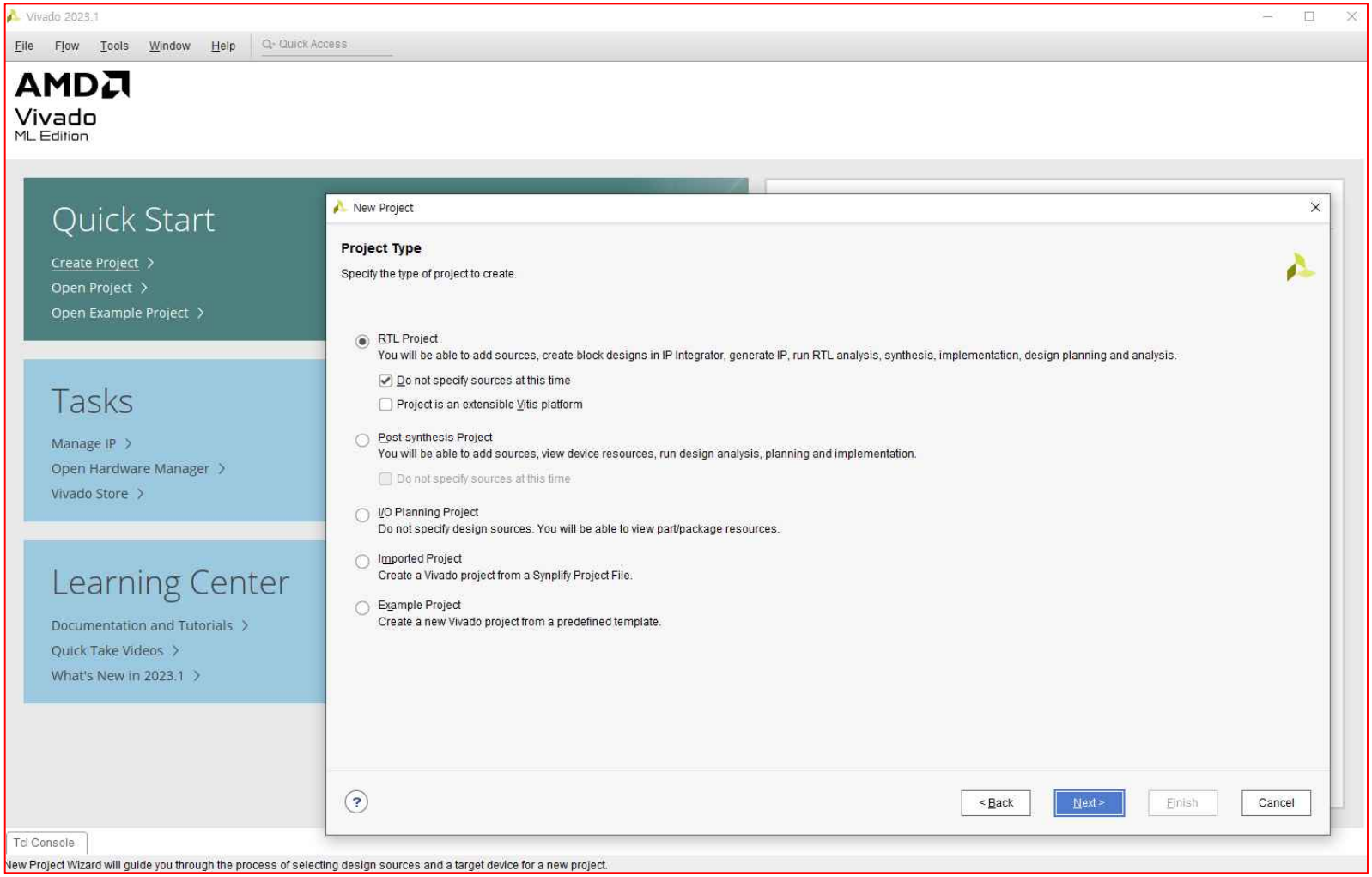
Hello World Implementation

- ***Hello World Implementation***
  - Vivado 2023.1 실행 → [Create Project] 클릭 → [Next]
  - Project name 및 위치 설정 → Next



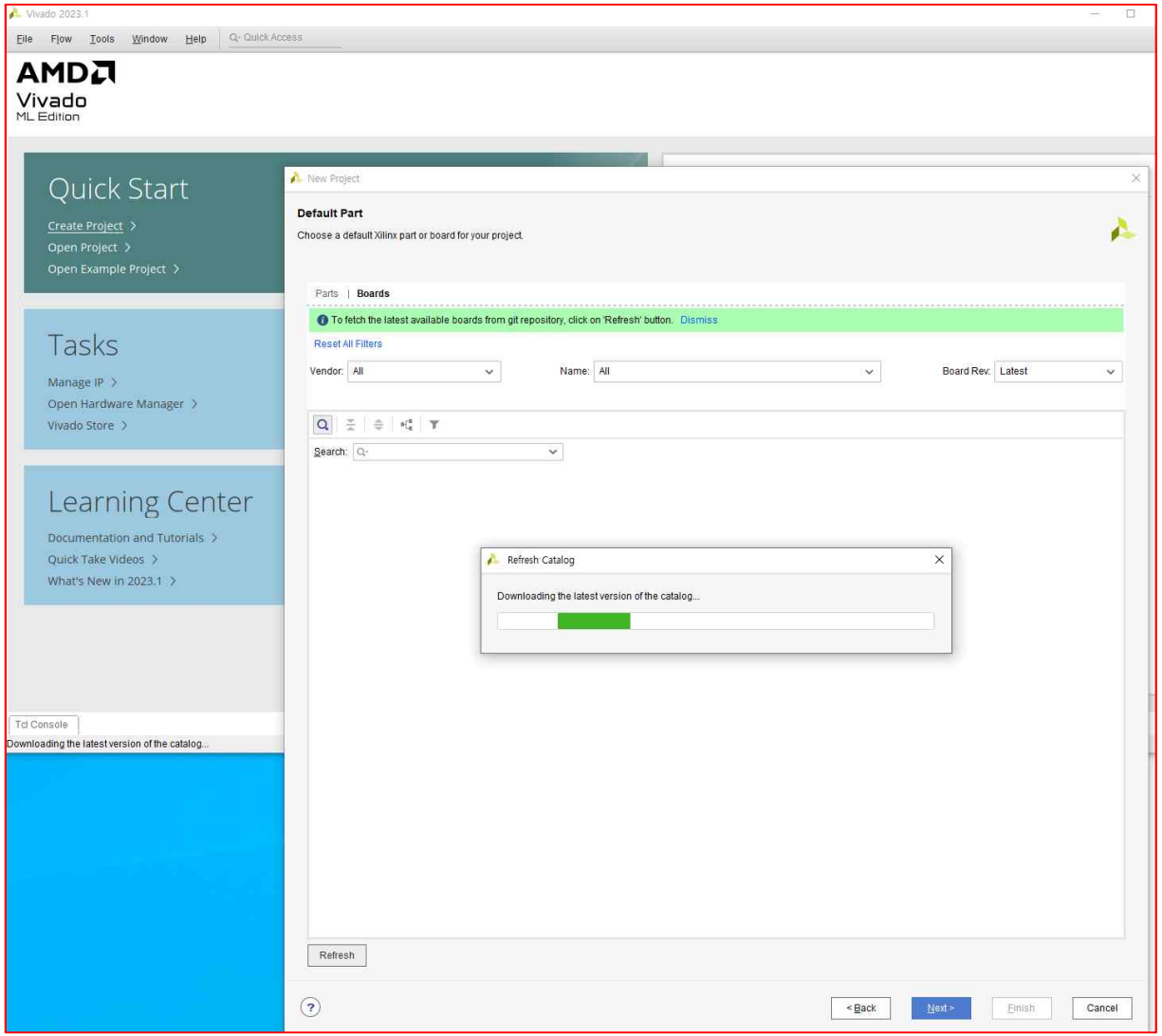
Hello World Implementation

- ***Hello World Implementation***
  - ***RTL Project → Next***



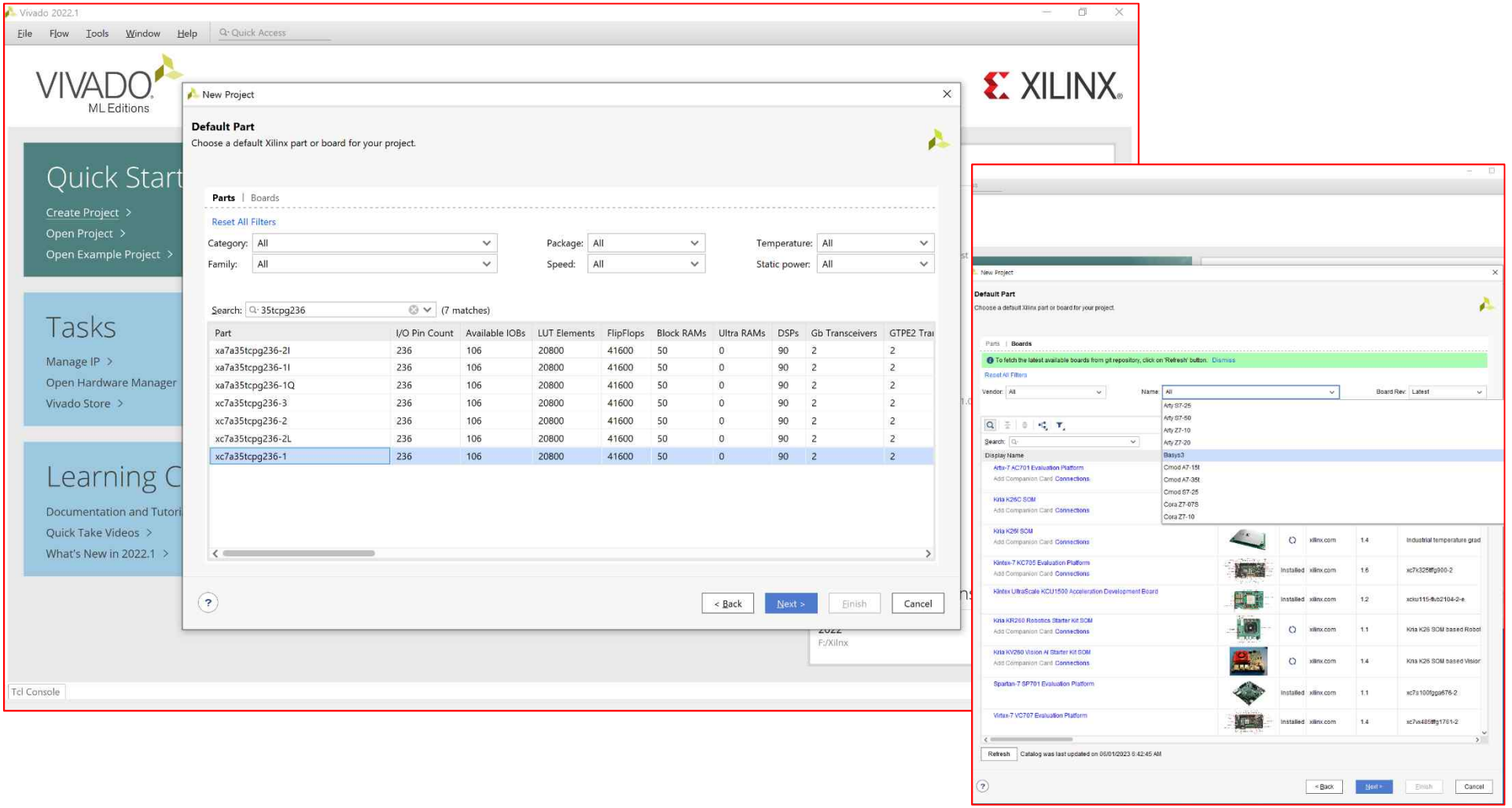
Hello World Implementation

- *Hello World Implementation*
  - *Default Part → Boards → Basys3 → Next*



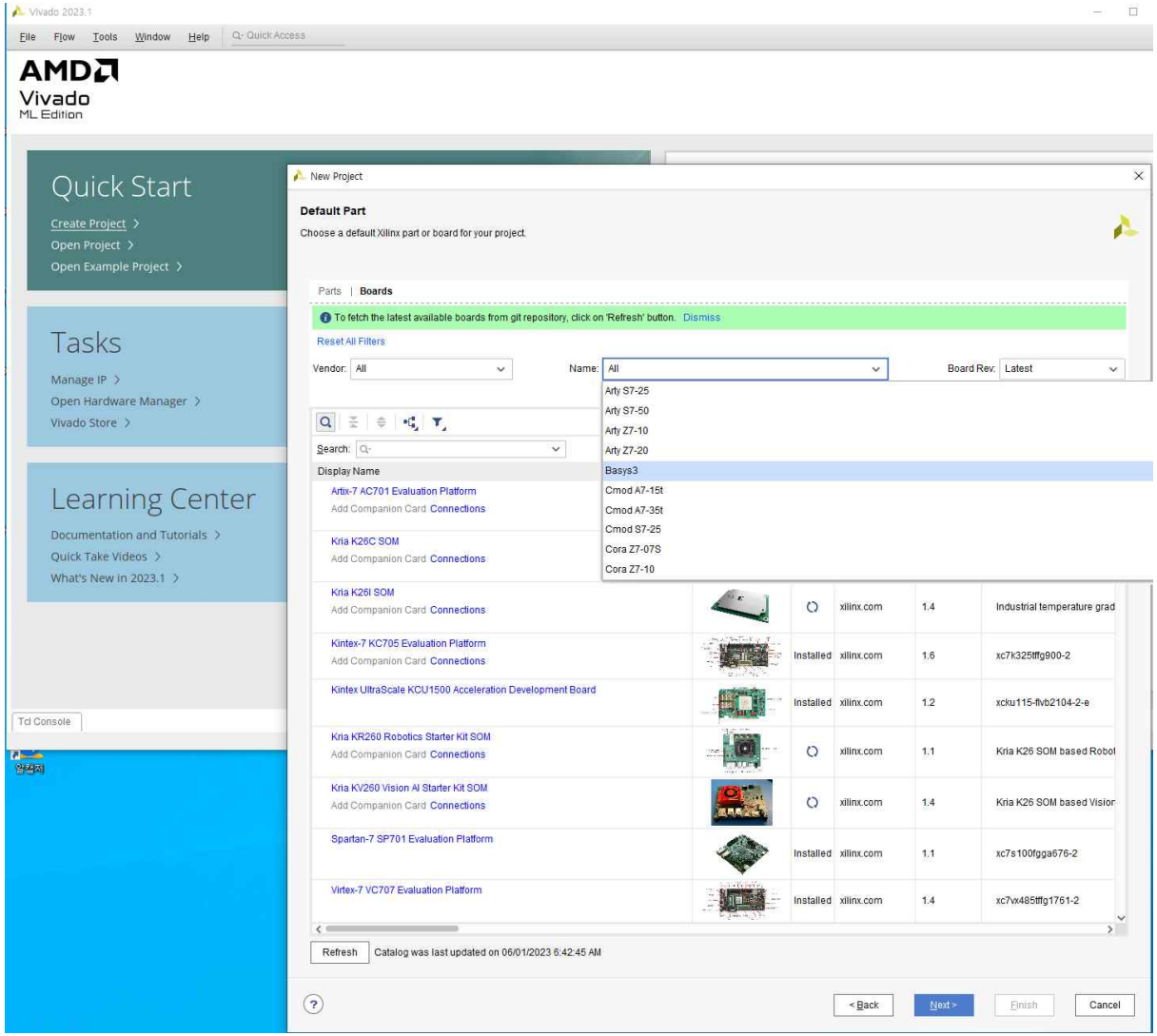
LED_Counter Implementation

- *LED_Counter Implementation ➔ Basic Template Implementation*
  - *Default Part ➔ Parts ➔ xc7a35tcpg236-1 ➔ Next*
    - ✓ *[or] Default Part ➔ Boards ➔ Basys3 ➔ Next*



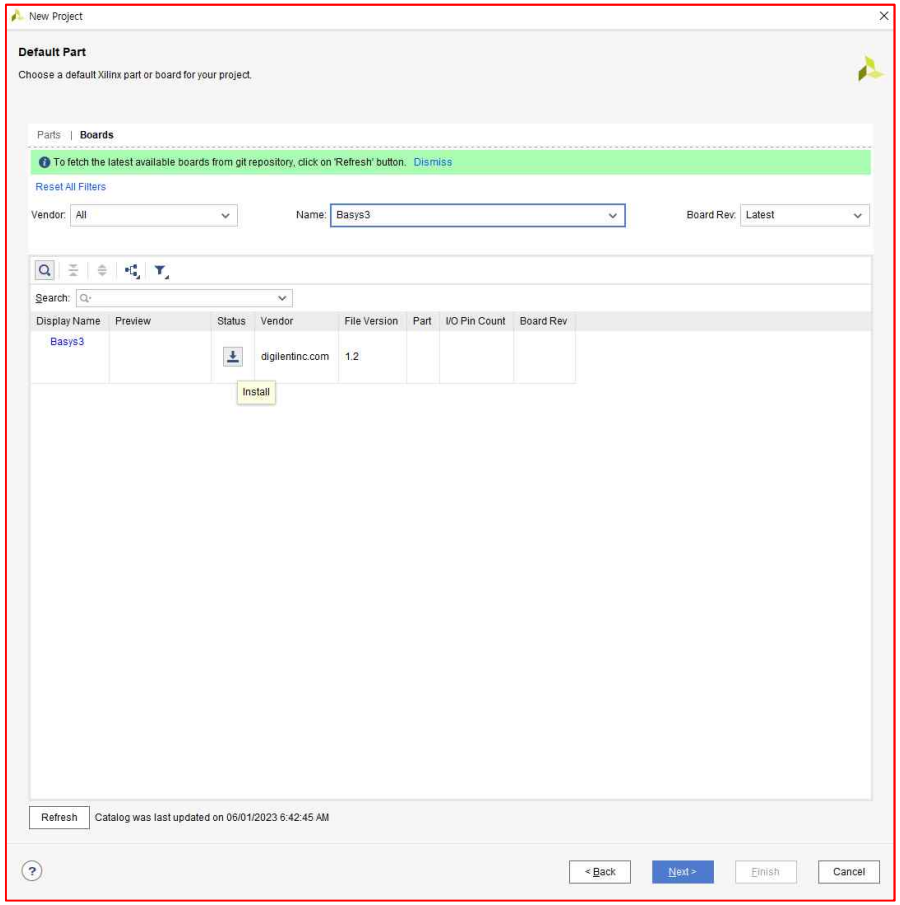
Hello World Implementation

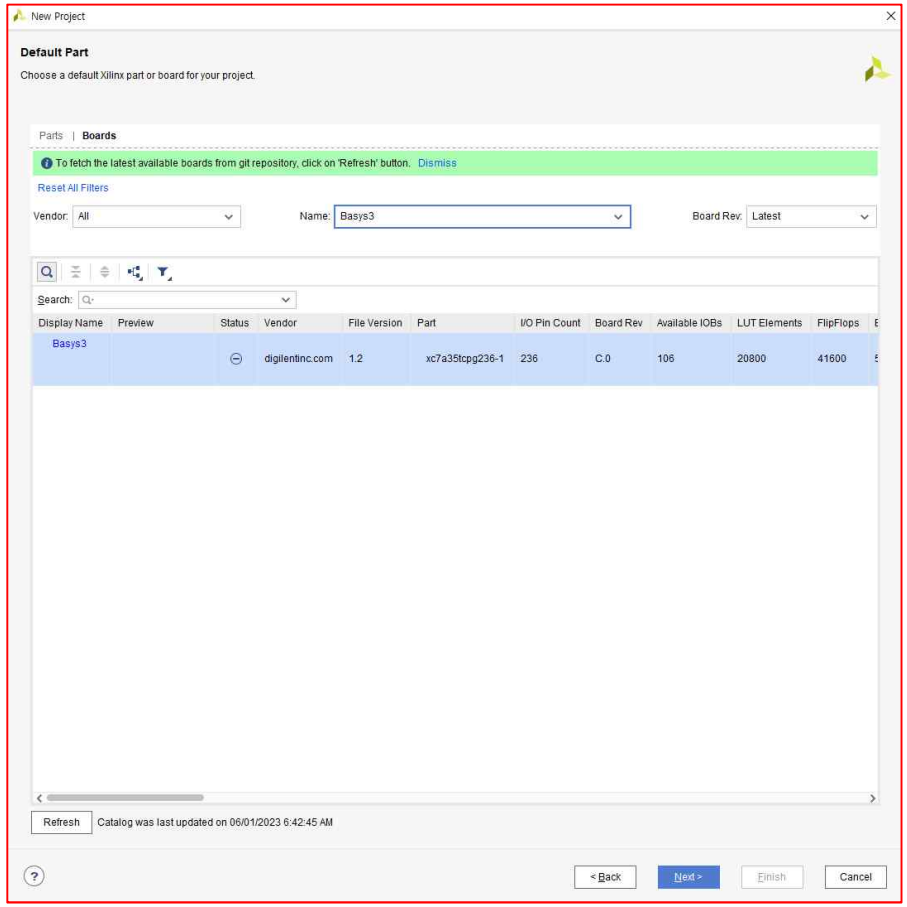
- ***Hello World Implementation***
  - ***Default Part ➔ Boards ➔ Basys3 ➔ Next***



Hello World Implementation

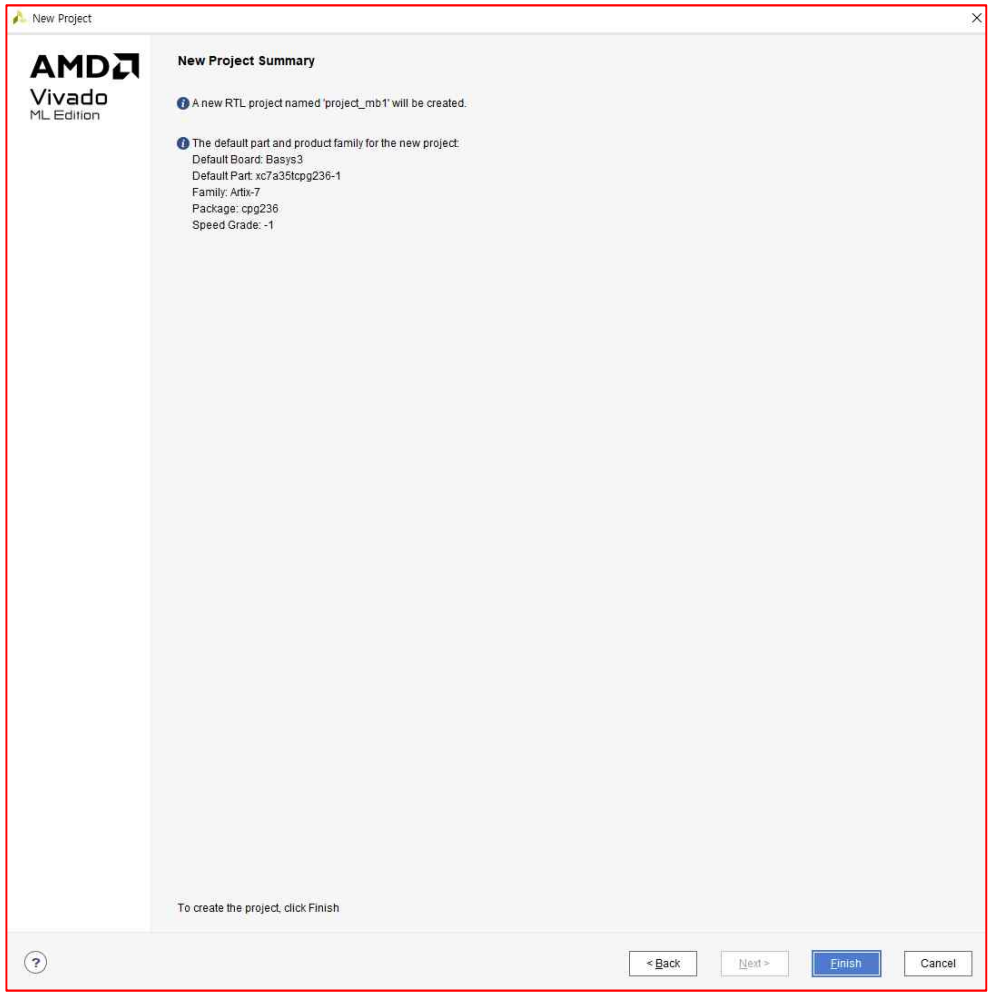
- *Hello World Implementation*
  - *Default Part → Boards → Basys3 → Next*





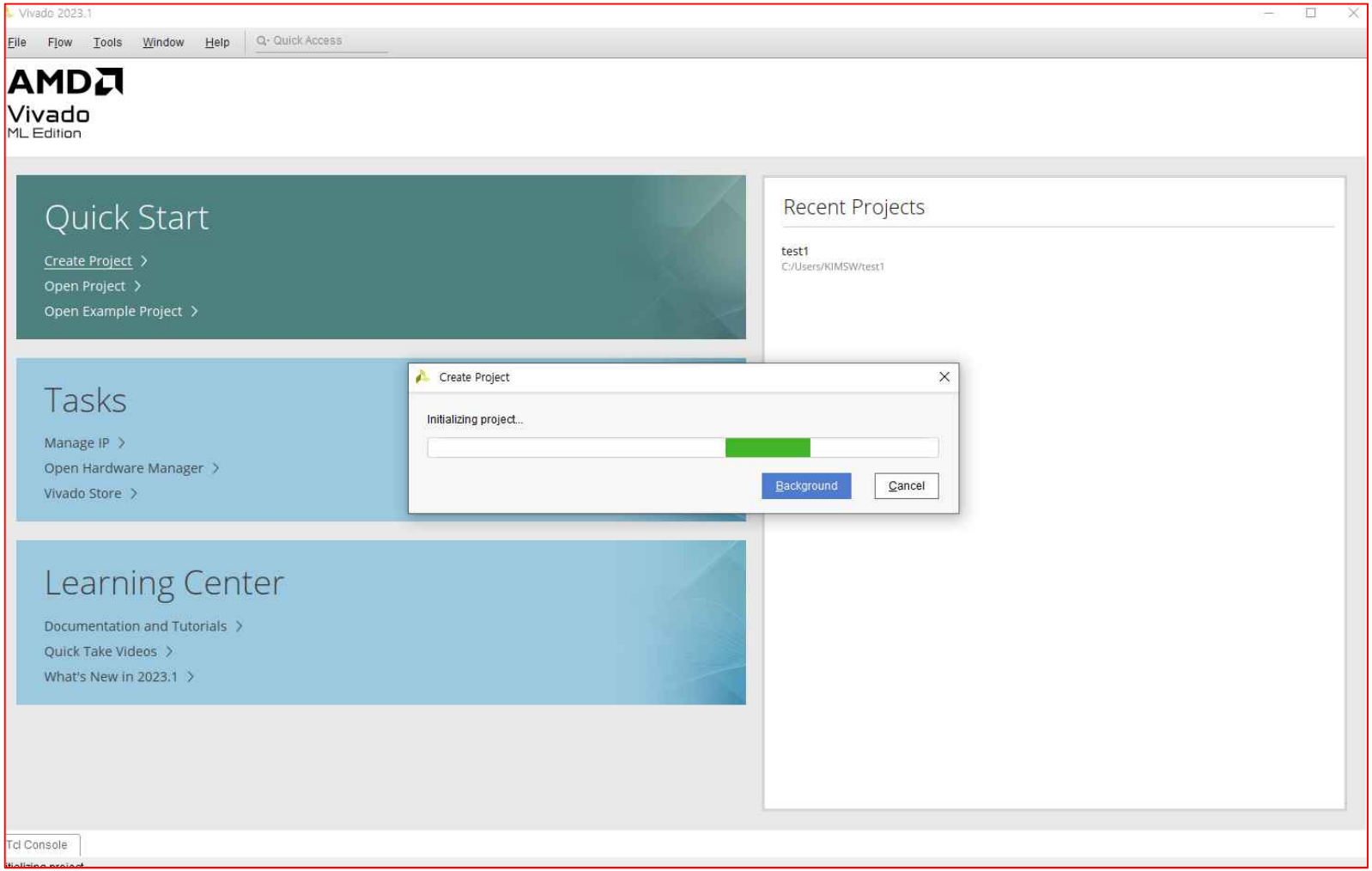
Hello World Implementation

- *Hello World Implementation*
  - *New Project Summary ➔ Finish*



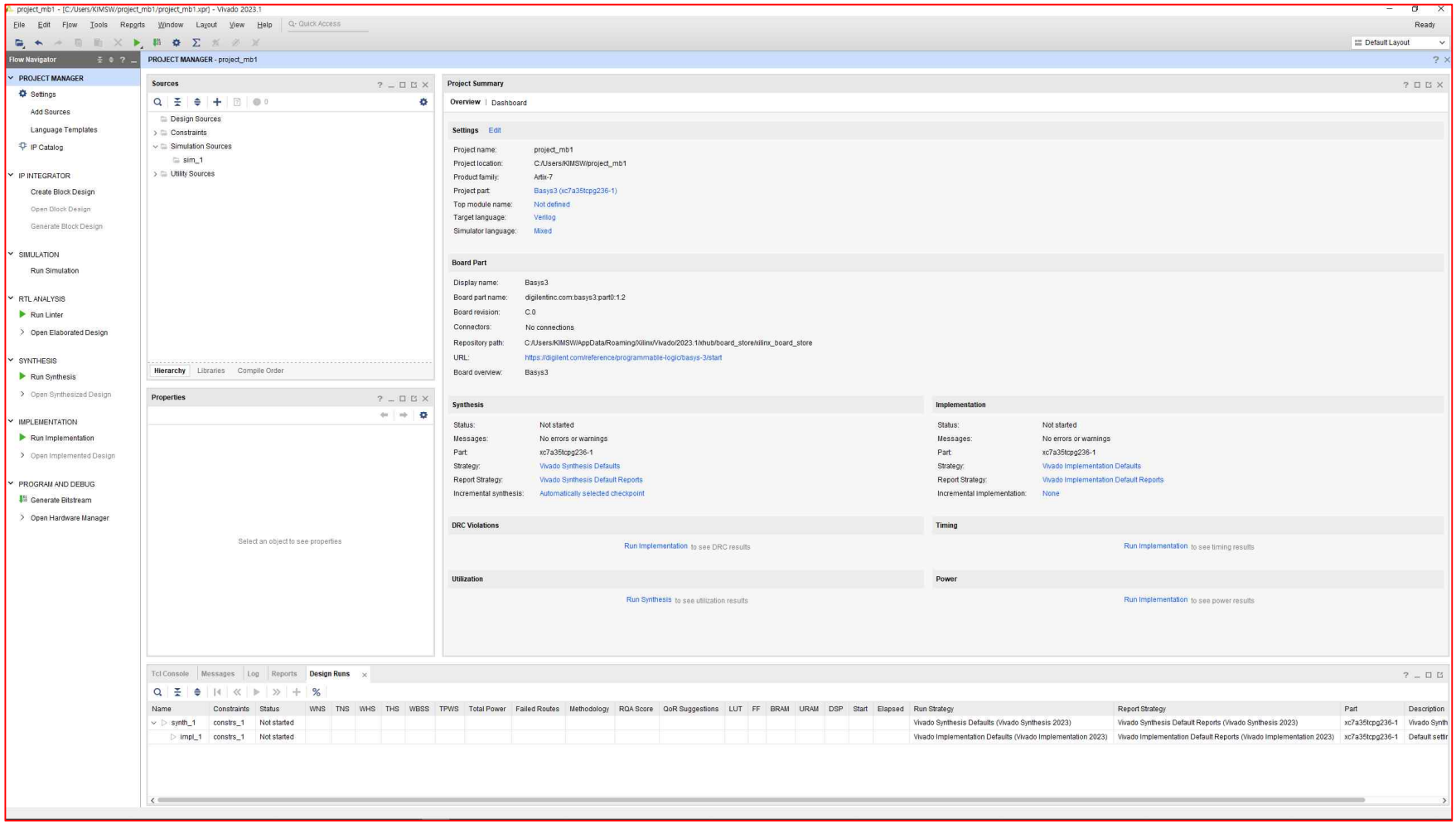
Hello World Implementation

- ***Hello World Implementation***
  - ***Create Project*** 완료



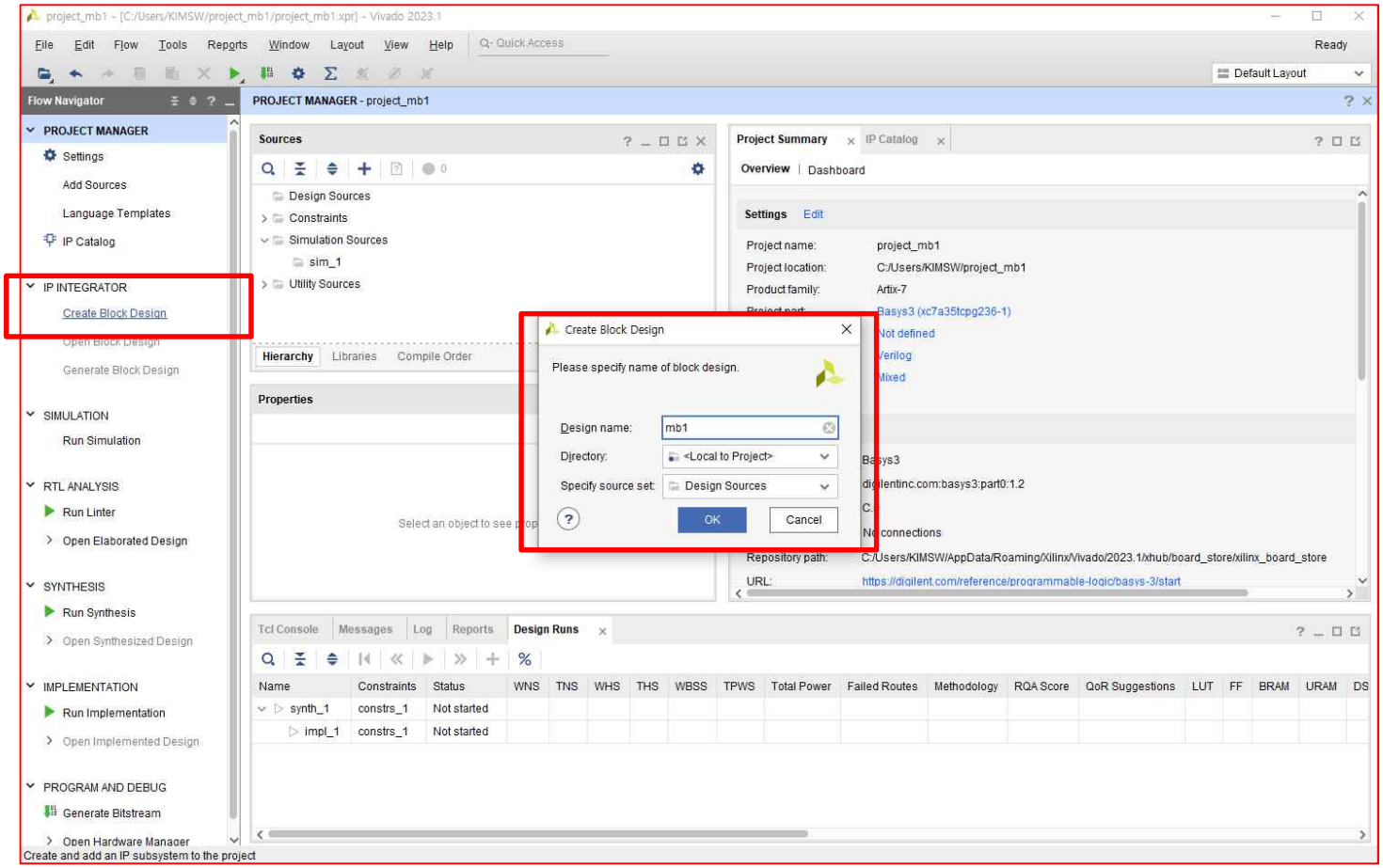
Hello World Implementation

- ***Hello World Implementation***
  - ***Create Project*** 완료 ➔ ***PROJECT MANAGER***



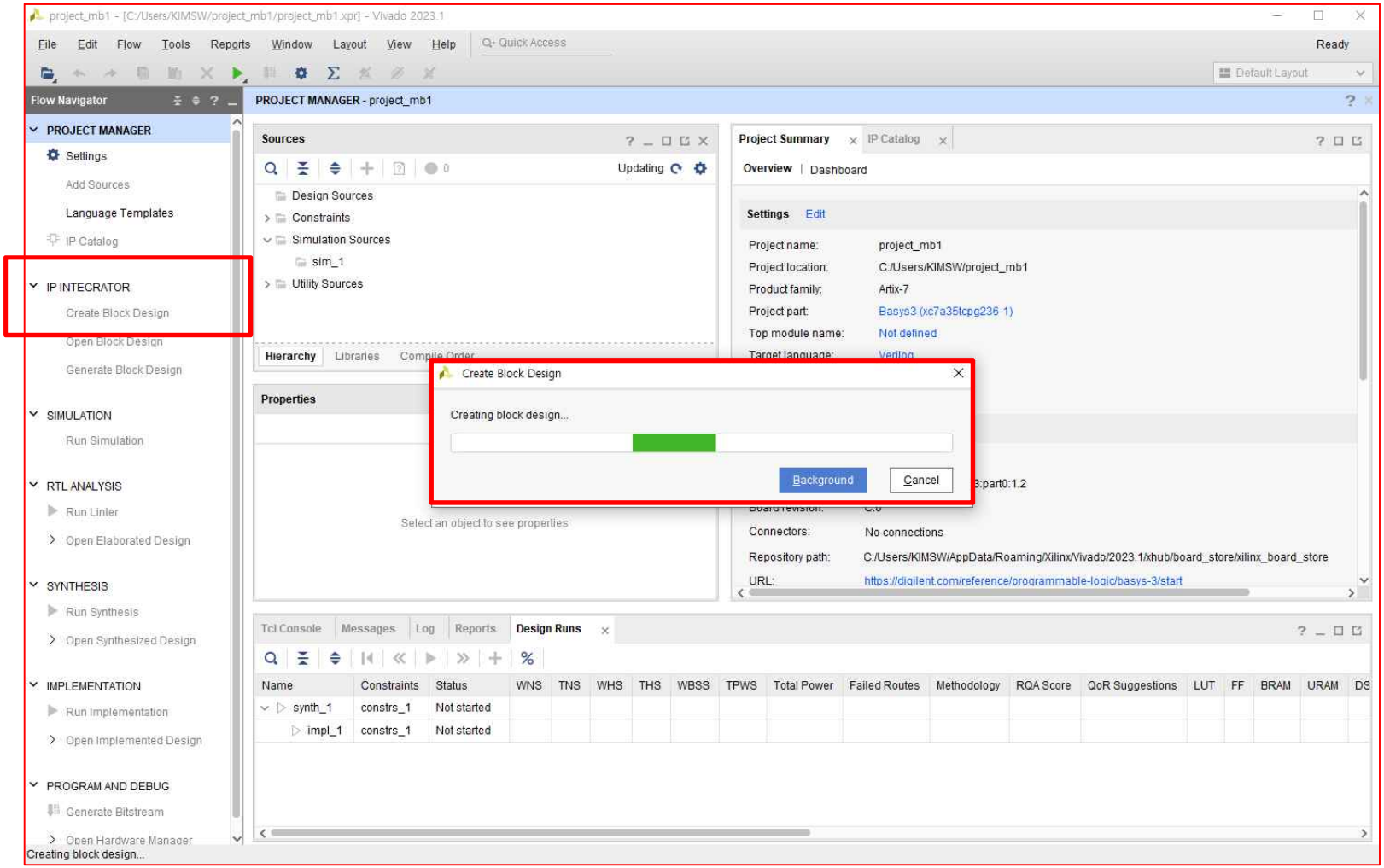
Hello World Implementation

- ***Hello World Implementation → Microblaze*** 생성
  - ***PROJECT MANAGER → IP INTEGRATOR → Create Block Design***
  - Microblaze : 사용자가 원하는 IP Block 을 선택 , 속성 지정 및 각 Block 들을 연결해서 사용
  - [***Design Name(ex. Microblaze_1)***] 입력 → ***OK***



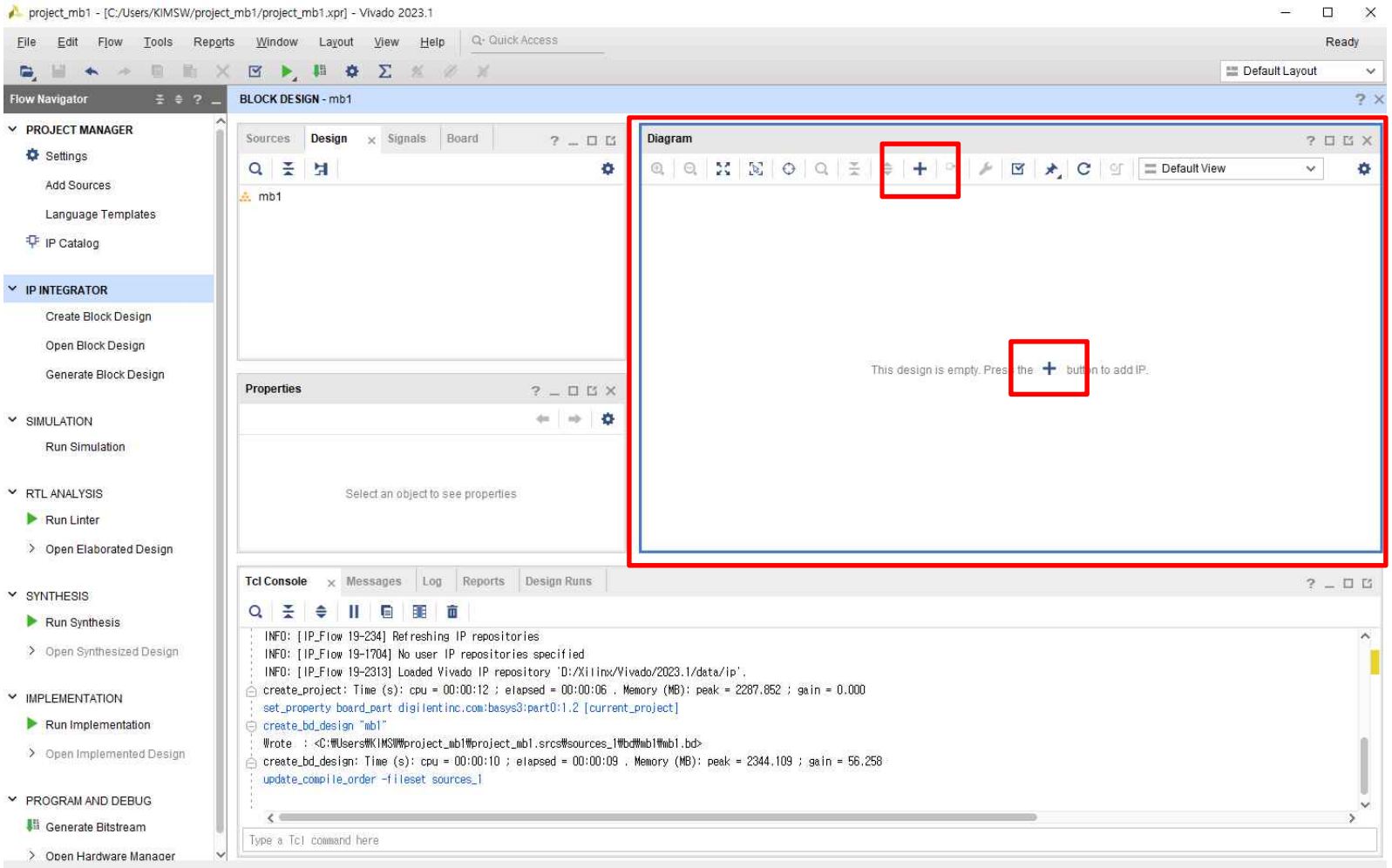
Hello World Implementation

- *Hello World Implementation → Microblaze 생성*
  - *IP INTEGRATOR → Create Block Design*



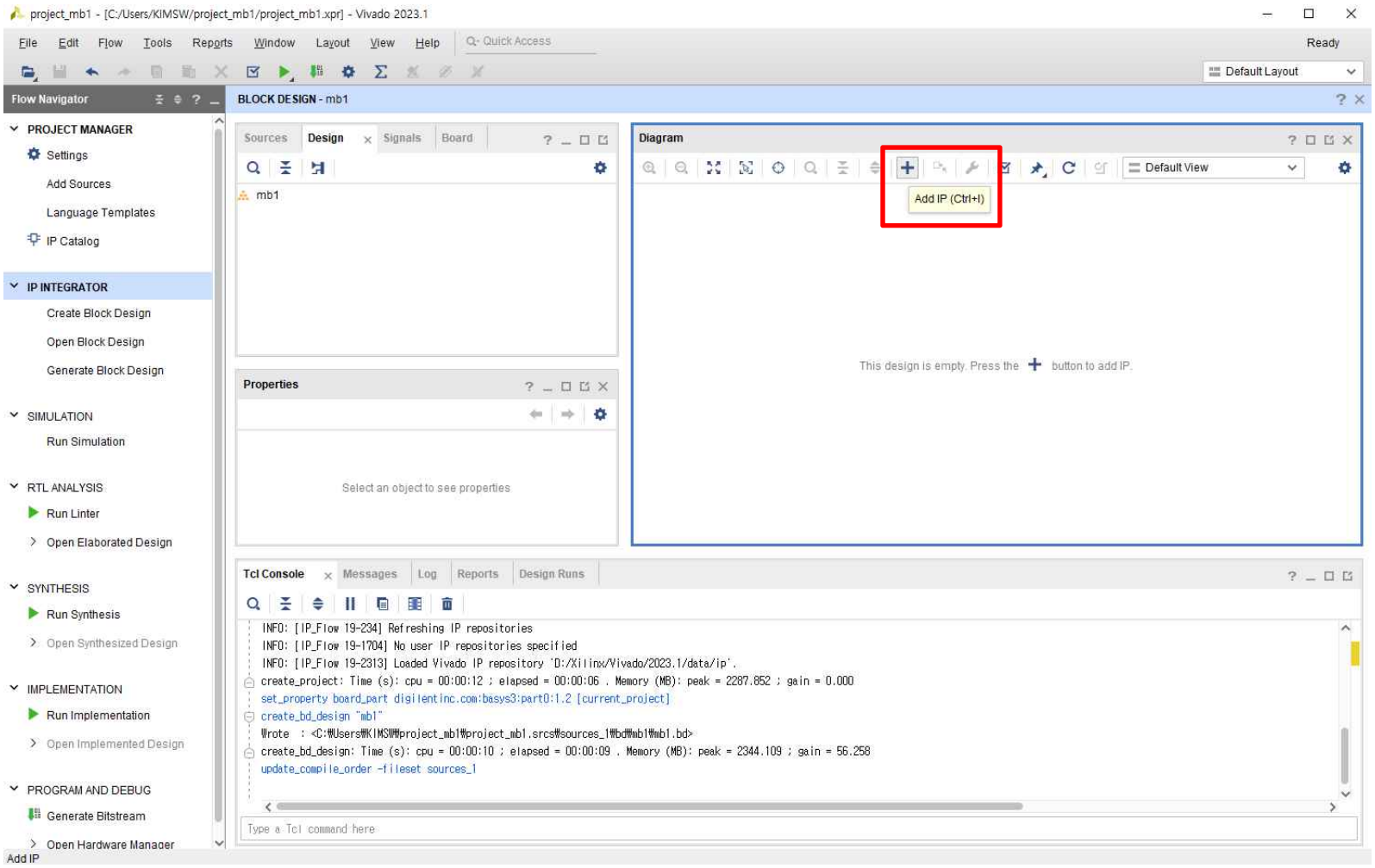
Hello World Implementation

- ***Hello World Implementation*** → ***Microblaze*** 생성
  - ***Diagram Window*** → ***Add IP*** → **“ + ”** 아이콘 클릭



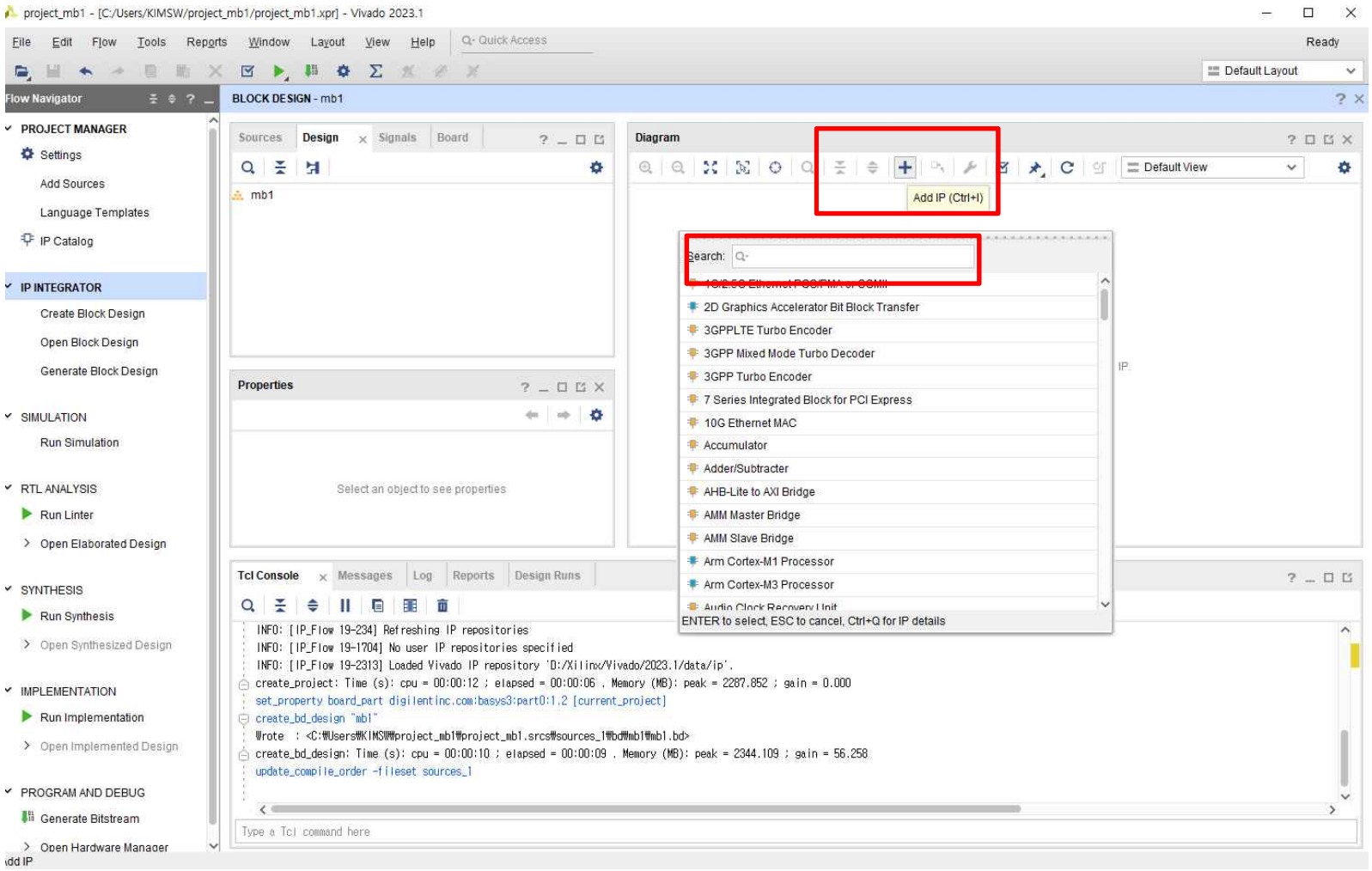
Hello World Implementation

- ***Hello World Implementation → Microblaze*** 생성
  - ***Diagram Window → Add IP →*** “+” 아이콘 클릭 → ***“MicroBlaze”*** 검색 및 ***IP [MicroBlaze]*** 추가(1)



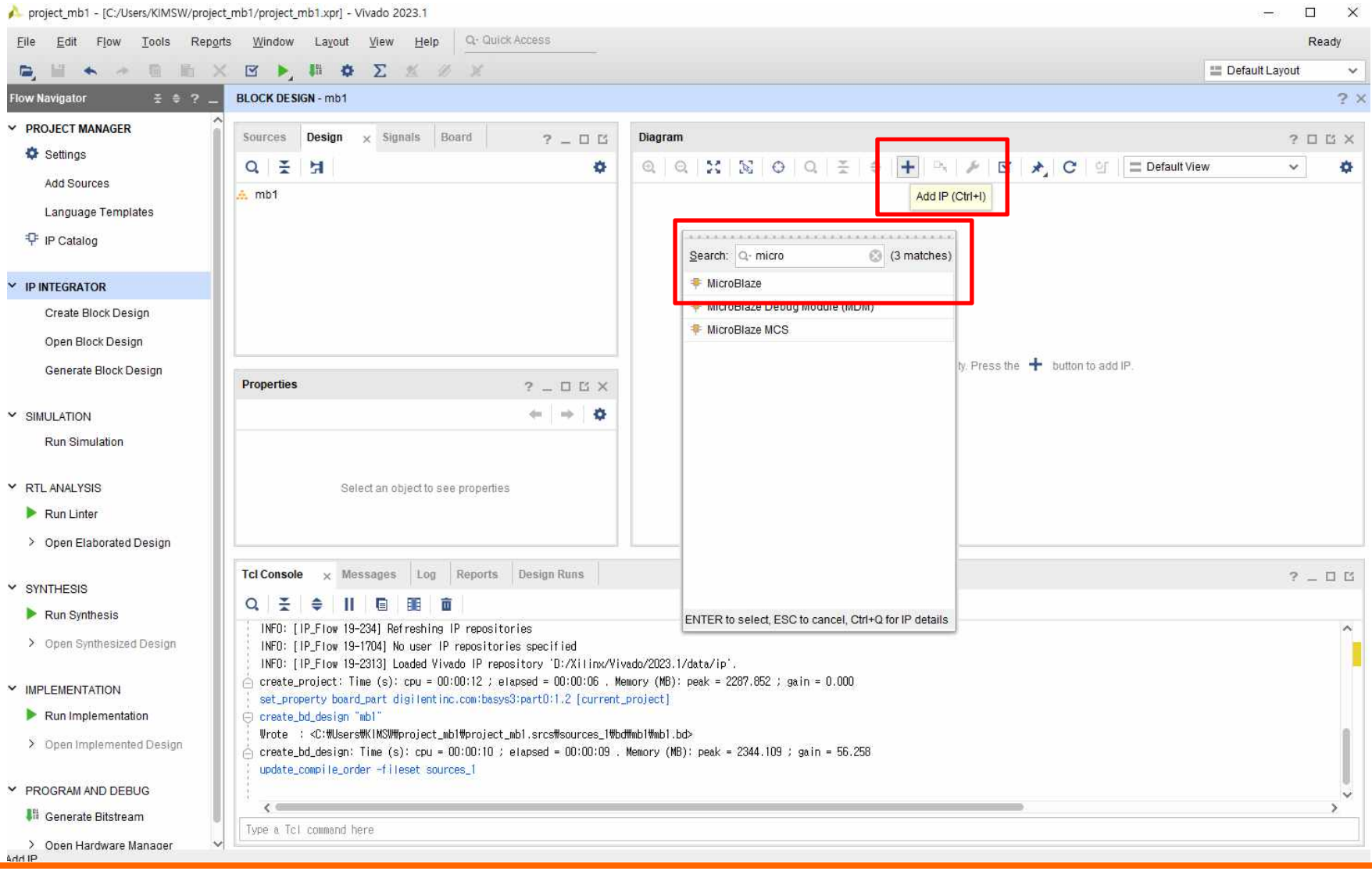
Hello World Implementation

- ***Hello World Implementation → Microblaze 생성***
  - ***Diagram Window → Add IP →*** “+” 아이콘 클릭 → ***“MicroBlaze” 검색 및 IP [MicroBlaze] 추가(2)***



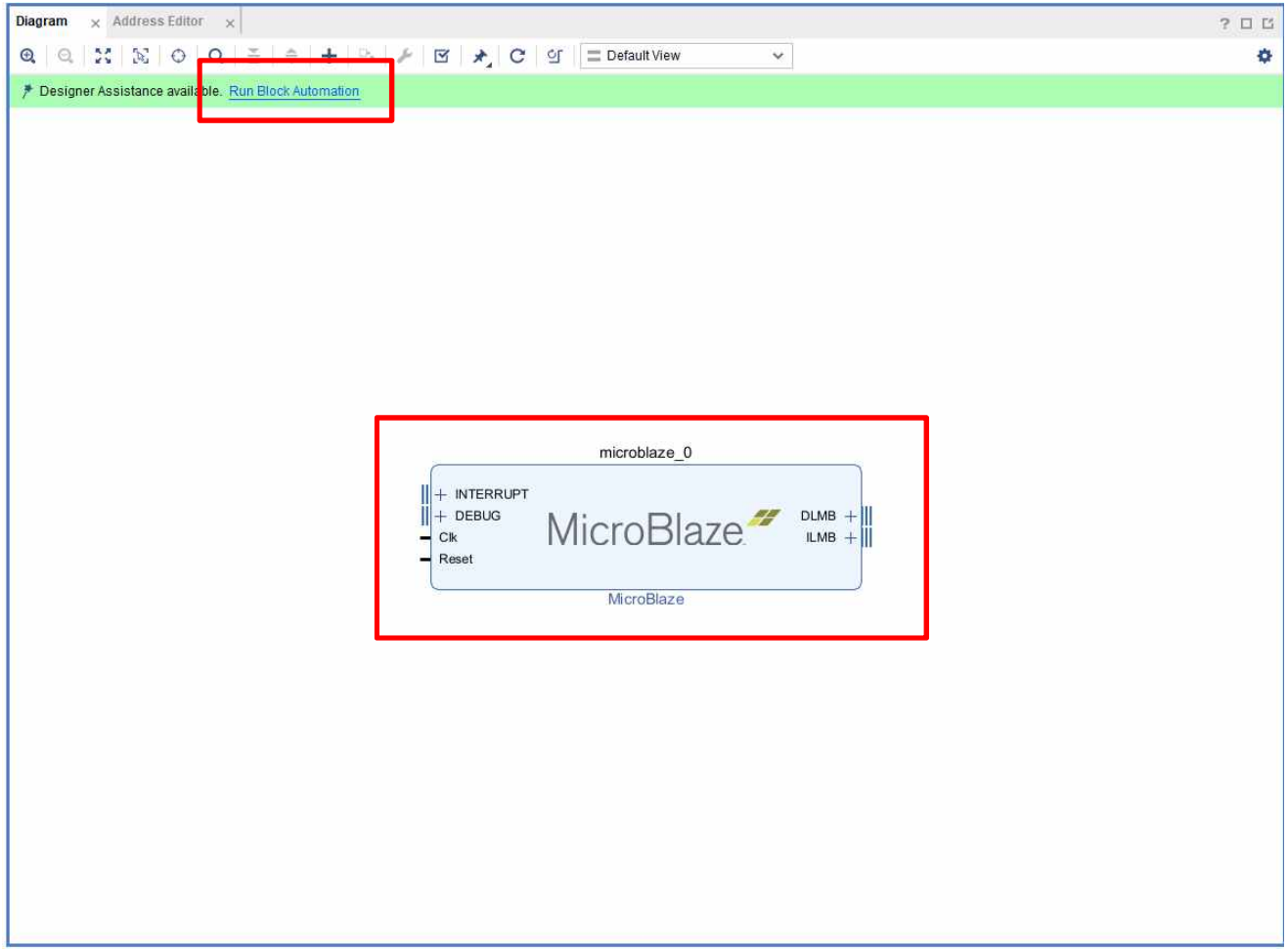
Hello World Implementation

- ***Hello World Implementation → Microblaze 생성***
  - ***Diagram Window → Add IP → "+" 아이콘 클릭 → "MicroBlaze" 검색 및 IP [MicroBlaze] 추가(3)***



Hello World Implementation

- *Hello World Implementation → Microblaze 생성*
  - *Diagram Window → Add IP →* “+” 아이콘 클릭 → *“MicroBlaze” 검색 및 IP [MicroBlaze] 추가(4)*
  - *Run Block Automation → Options*

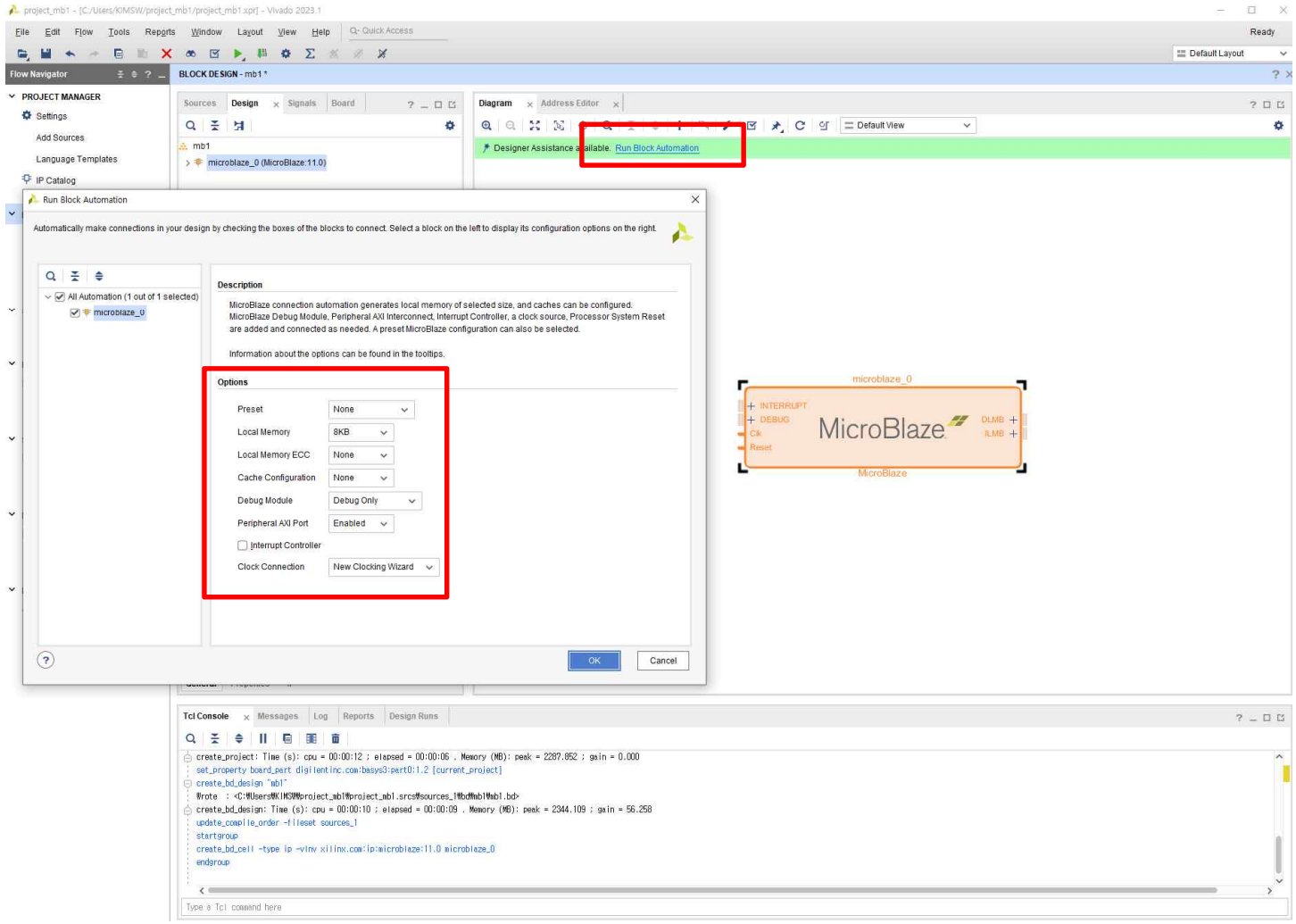


Hello World Implementation

➢ *Hello World Implementation* ➔ *Microblaze* 생성

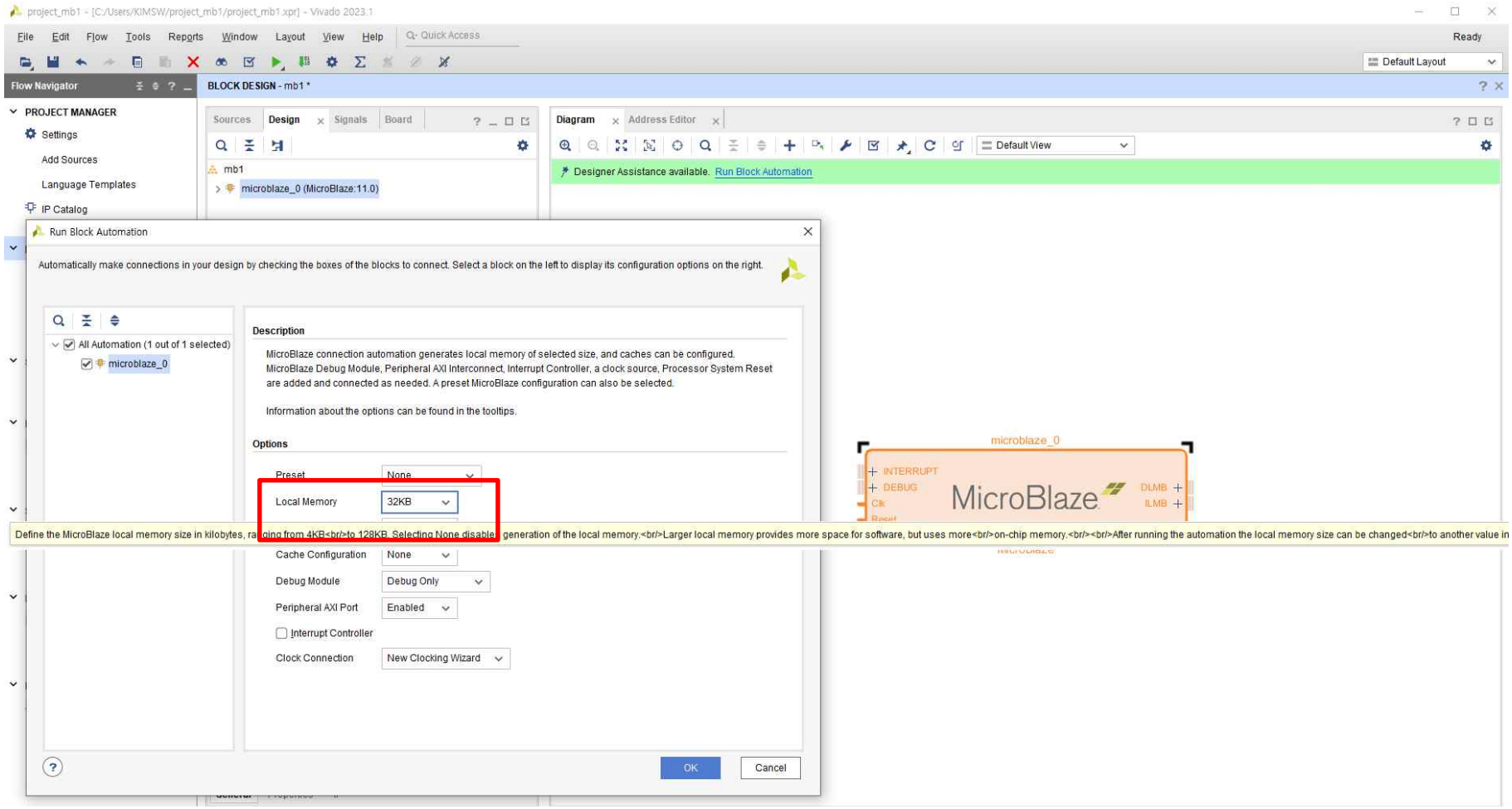
- *Run Block Automation* ➔ *Options*

☞ *[Local Memory : Code Memory]* 설정 ➔ OK



Hello World Implementation

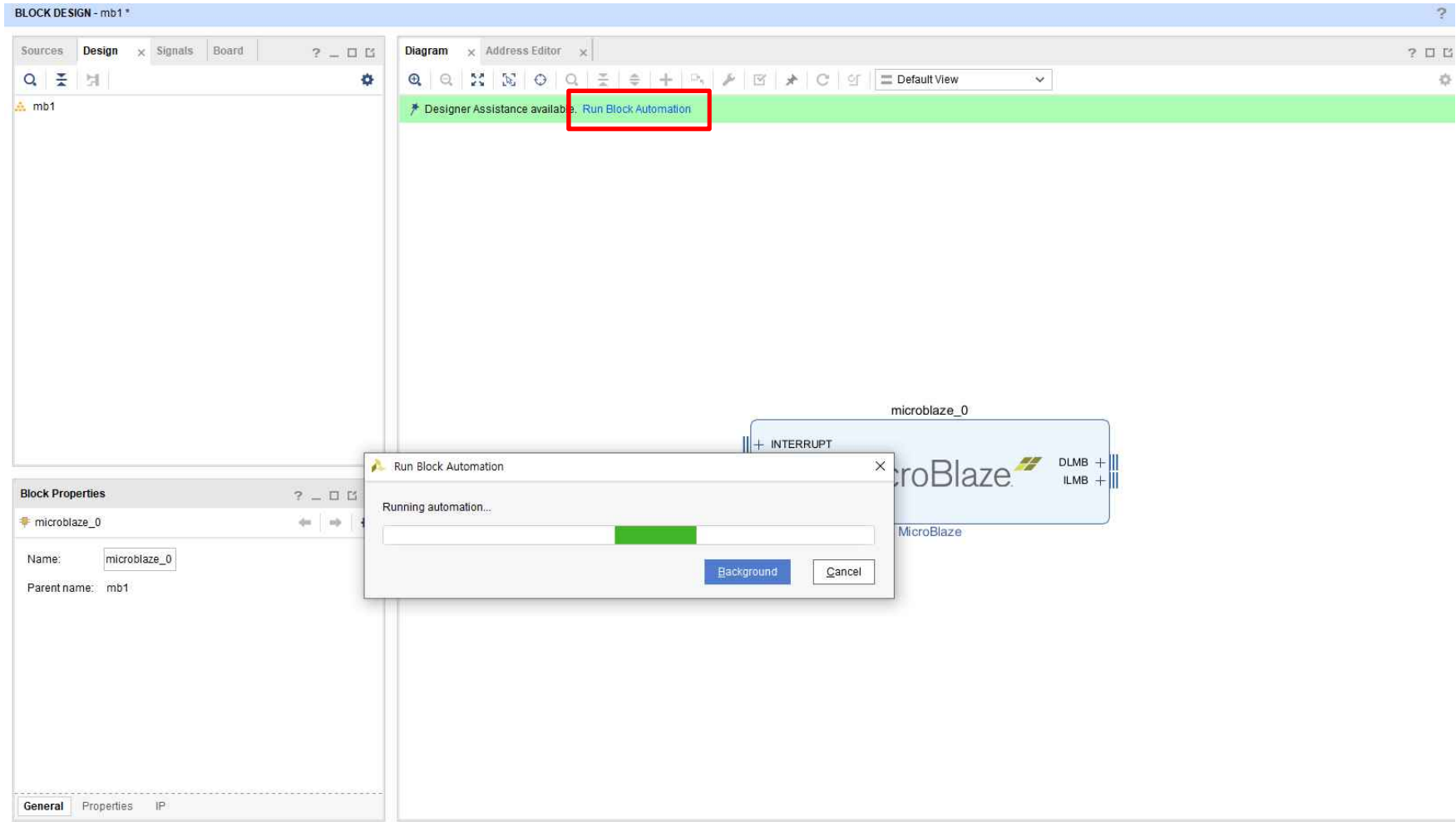
- ***Hello World Implementation*** → ***Microblaze*** 생성
  - ***Run Block Automation*** → ***Options***
    - ☞ ***[Local Memory : Code Memory]*** 설정 → OK



Hello World Implementation

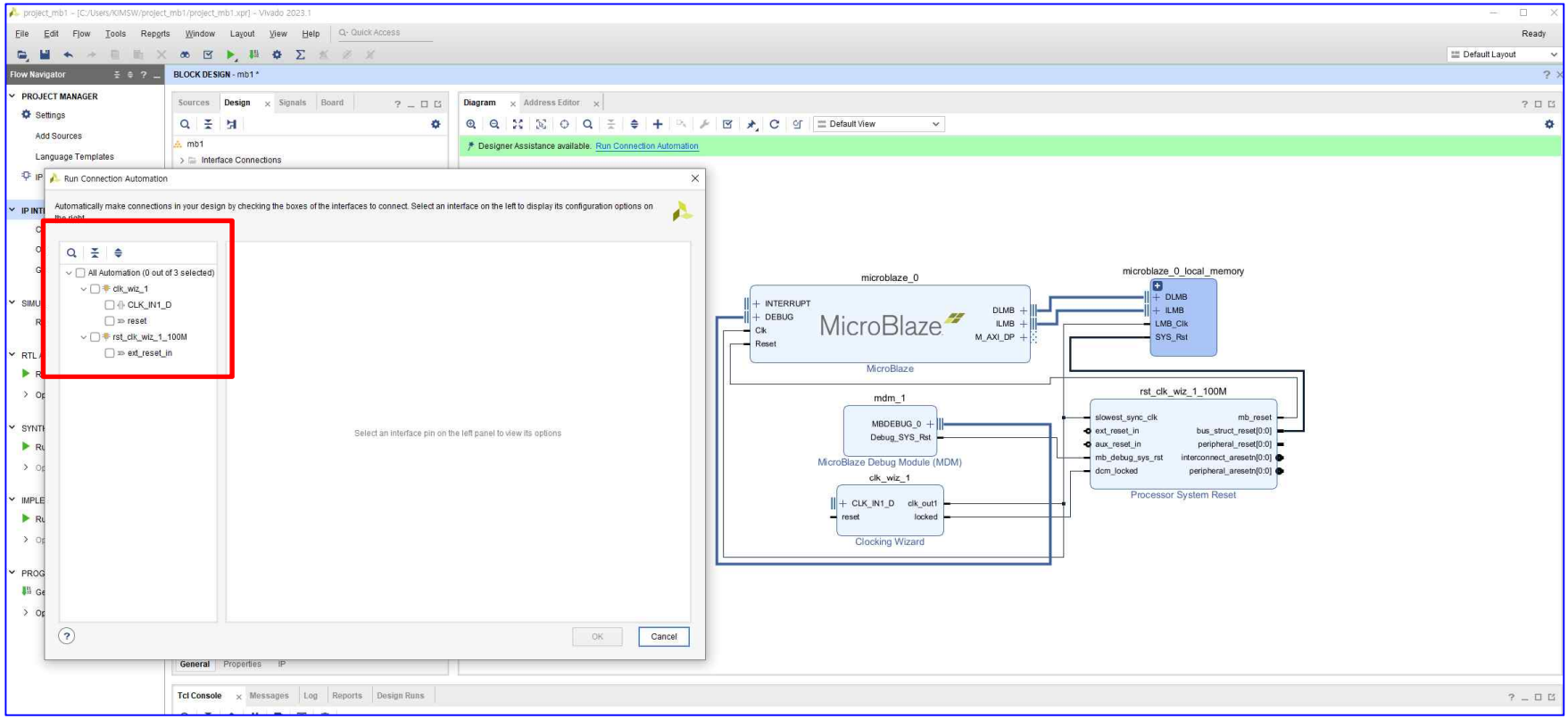
➢ ***Hello World Implementation***

- ***Run Connection Automation*** ➔ ***Block*** 자동 연결
- ***IP*** 추가 및 수정 후 ➔***[Run Block Automation] & [Run Connection Automation]*** 기능 적용



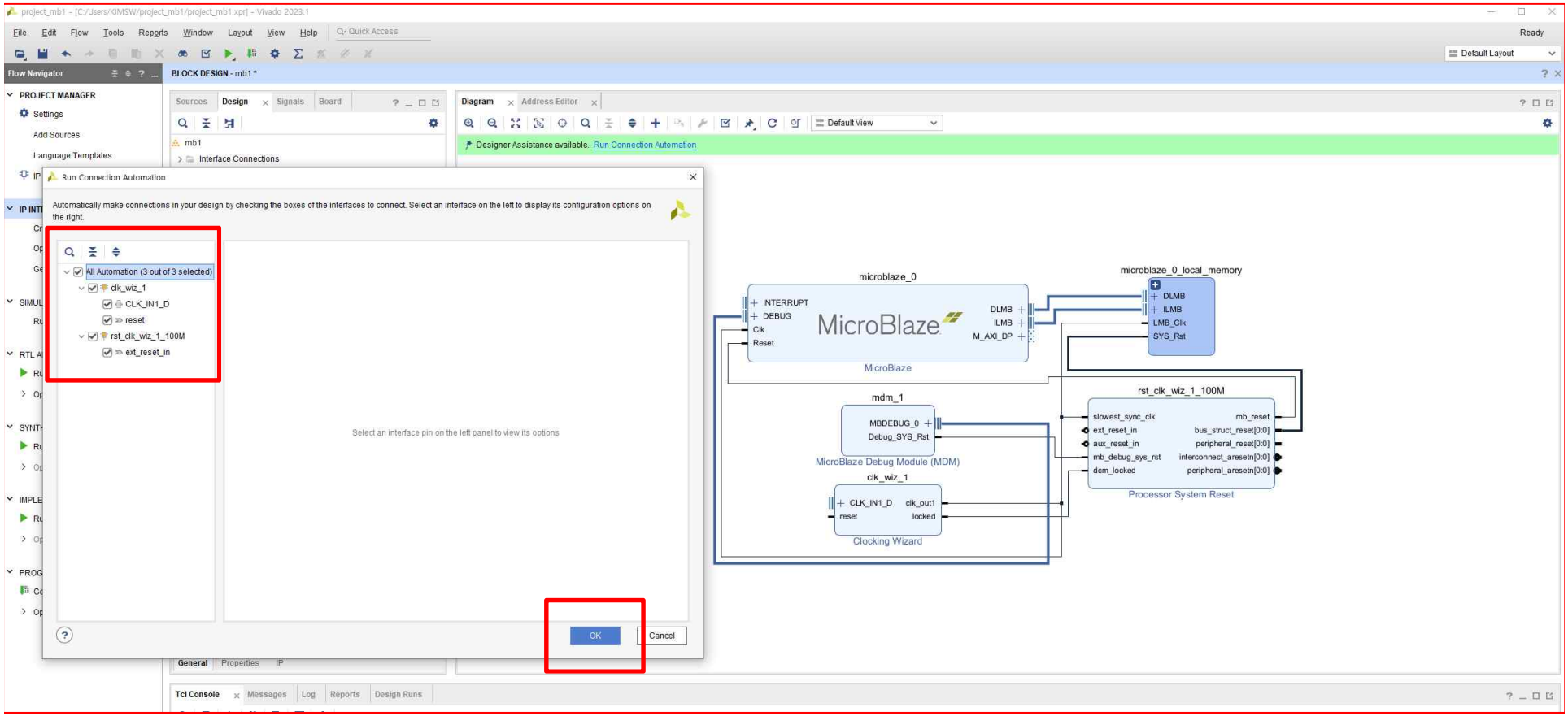
Hello World Implementation

- ***Hello World Implementation***
  - ***Run Connection Automation*** ➔ ***Block*** 자동 연결
  - ***[All Automation ➔ OK]*** 생성된 ***Block*** 자동 연결



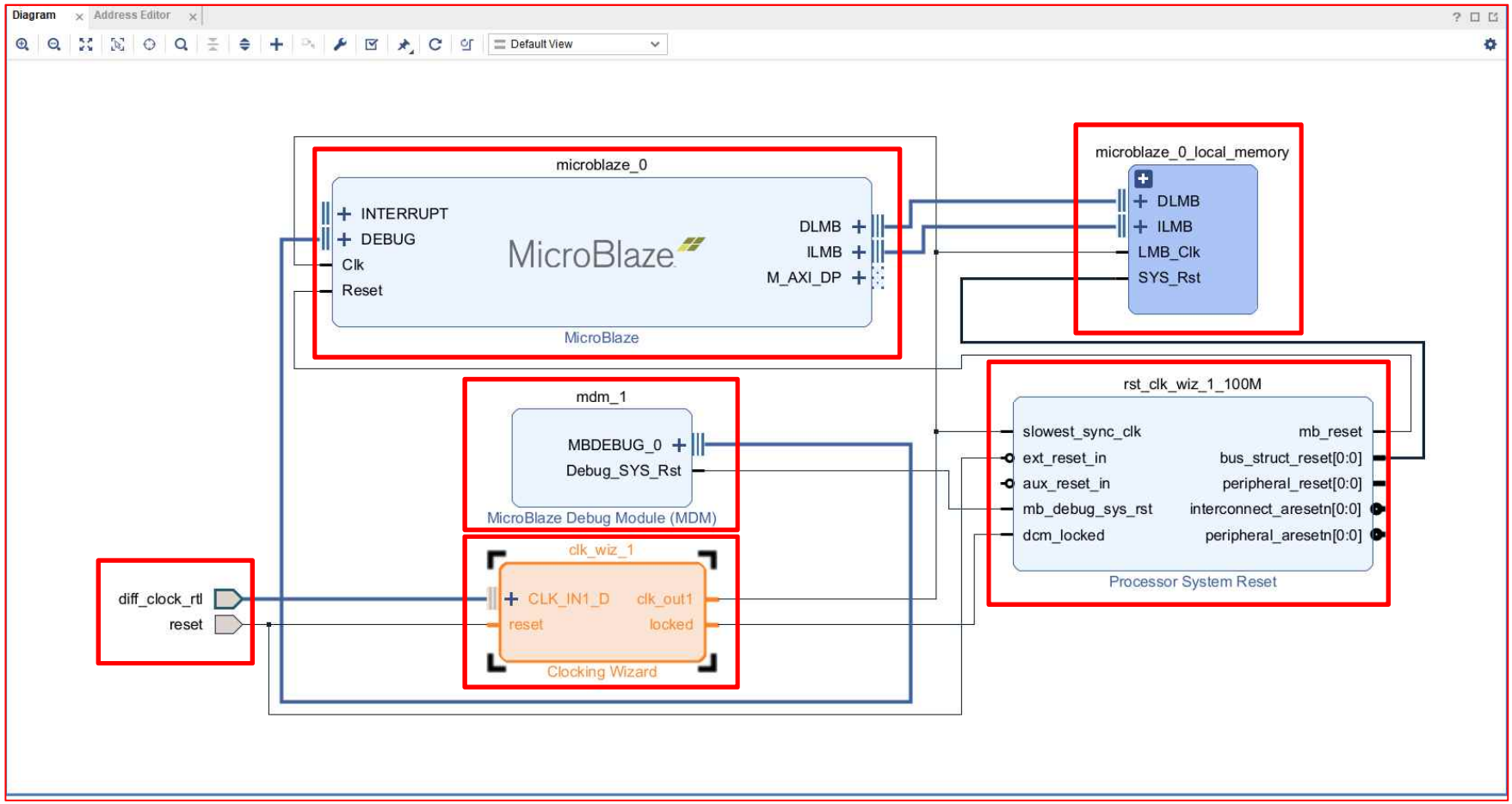
Hello World Implementation

- ***Hello World Implementation***
  - ***Run Connection Automation*** ➔ ***Block*** 자동 연결
  - ***[All Automation ➔ OK]*** 생성된 Block 자동 연결



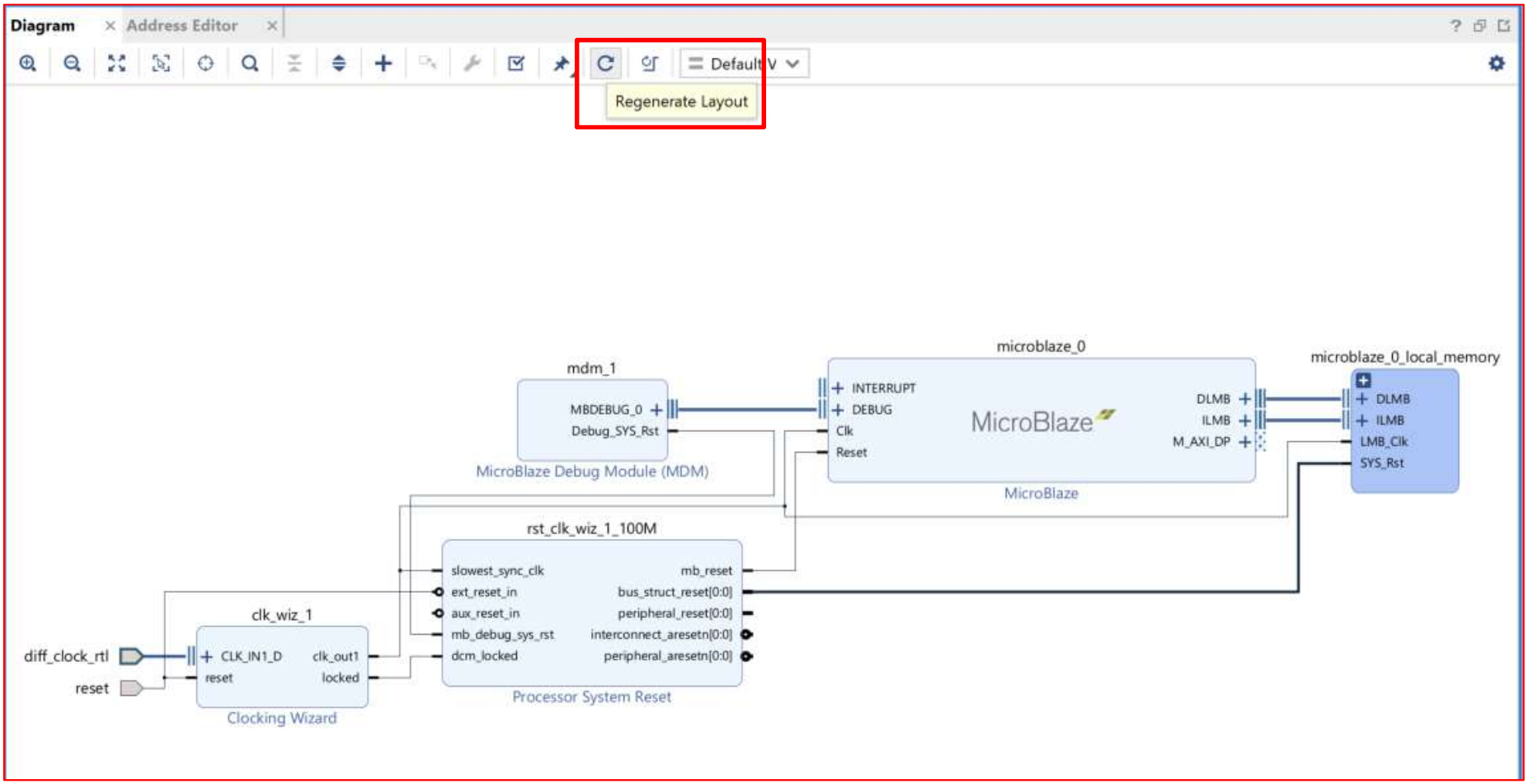
Hello World Implementation

- ***Hello World Implementation*** → ***Microblaze*** 생성
  - ***[MicroBlaze Core // Clock // Reset // Local Memory // Debug Module]*** 생성



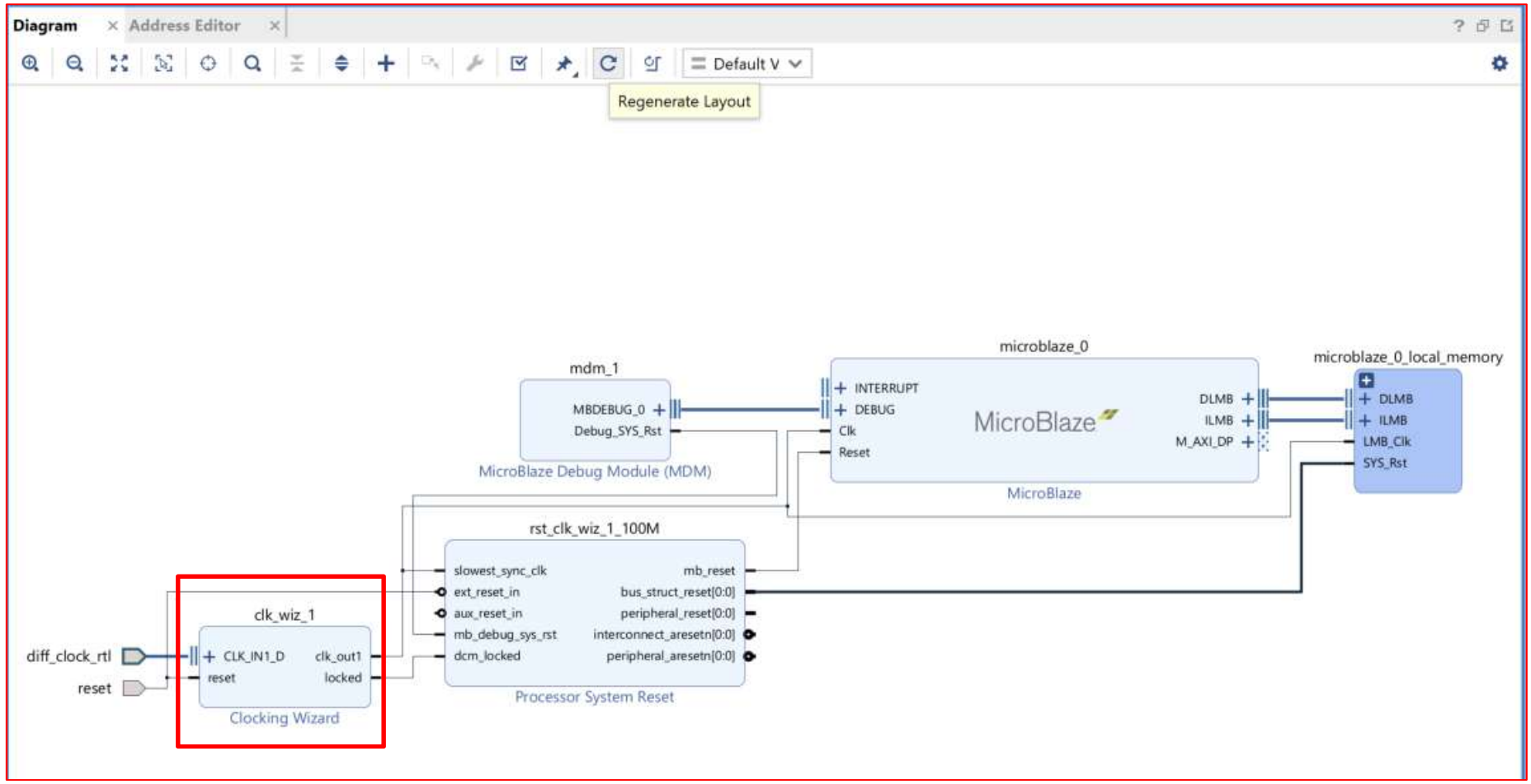
Hello World Implementation

- ***Hello World Implementation***
  - 배치 및 정리 ➔ ***Regenerate Layout*** 클릭



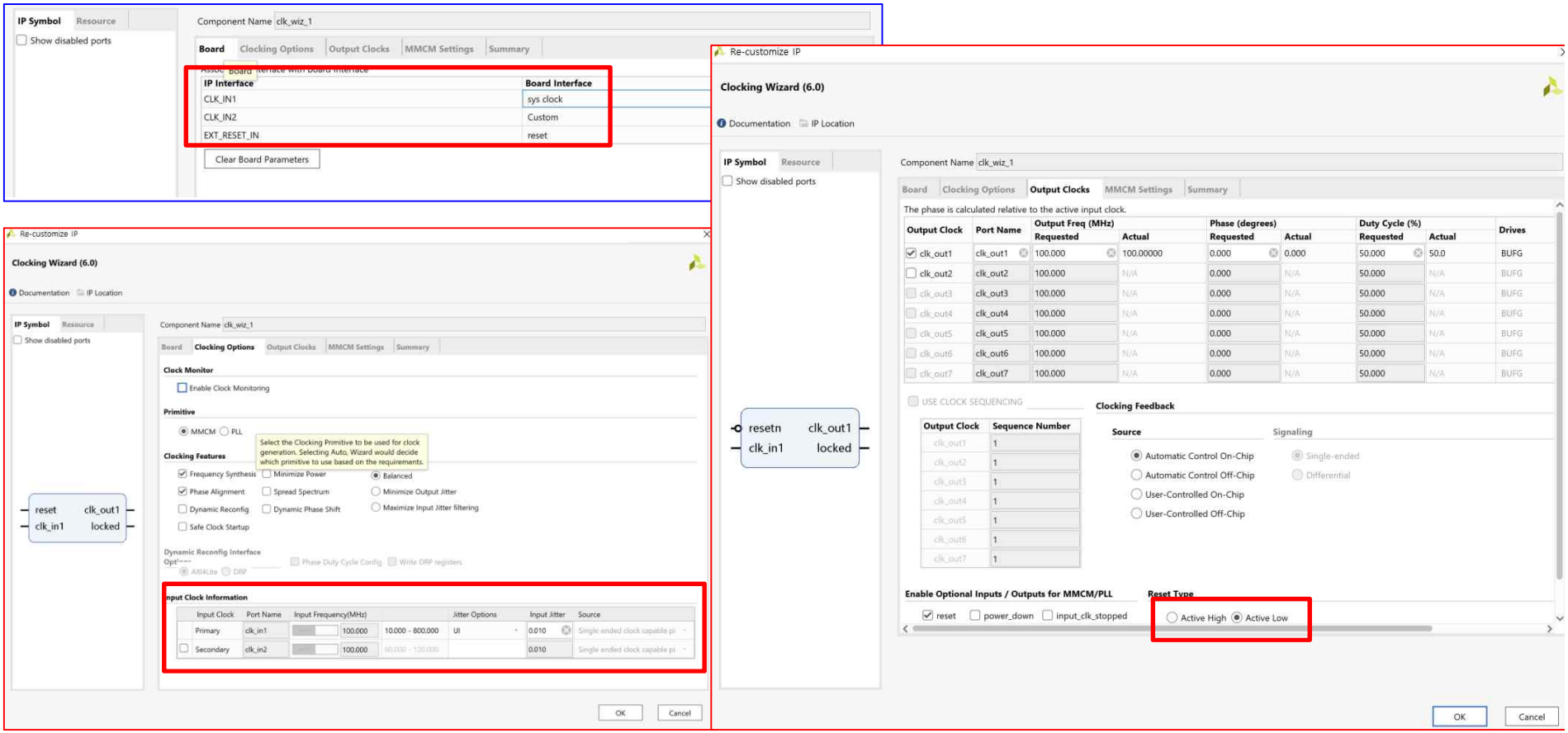
Hello World Implementation

- ***Hello World Implementation***
  - ***Basys3(Artix 7) HW***를 위한 속성 변경
    - ***clk_wiz_1*** 속성 변경 ➔ ***Double Click***



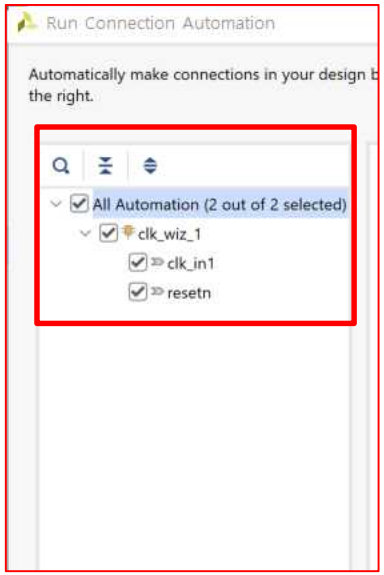
Hello World Implementation

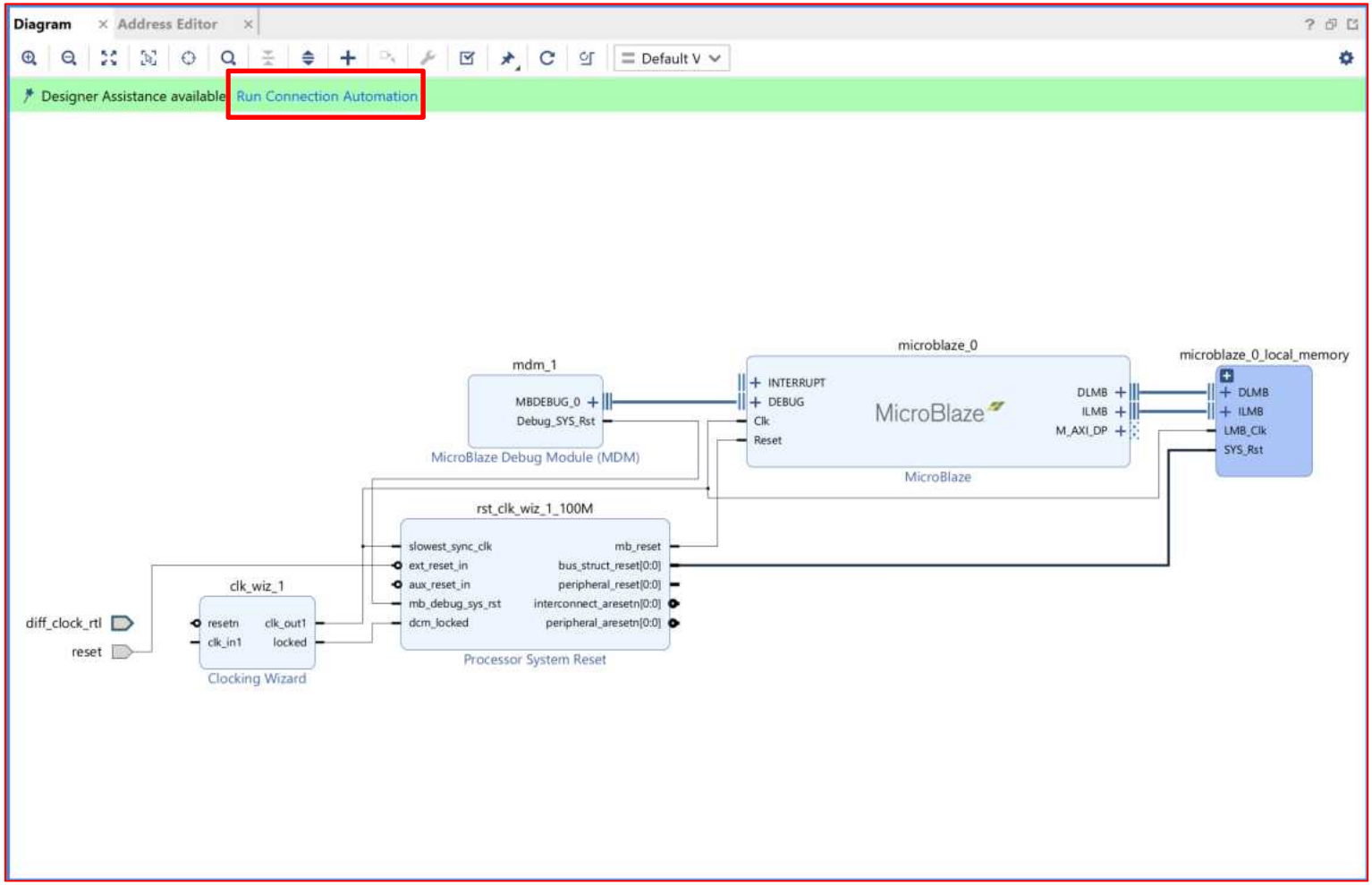
- ***Hello World Implementation*** → ***Microblaze*** 생성
  - ***clk_wiz_1 Block***을 선택 후 Double 클릭 → 속성 변경
  - ***Board*** 탭 → ***CLK_IN1 : sys_clock***
  - ***Clocking Options*** 탭 → ***clk_in1 : Single ended clock*** [← ***sys_clock*** 설정시 자동 설정]
  - ***Output Clocks*** 탭 → ***Reset Type : Active Low (or Active High)***



Hello World Implementation

- ***Hello World Implementation*** → ***Microblaze*** 생성
  - ***clk_wiz_1 Block***을 선택 후 ***Double*** 클릭 → 속성 변경
  - 속성 변경 완료 후 → ***Run Connection Automation***

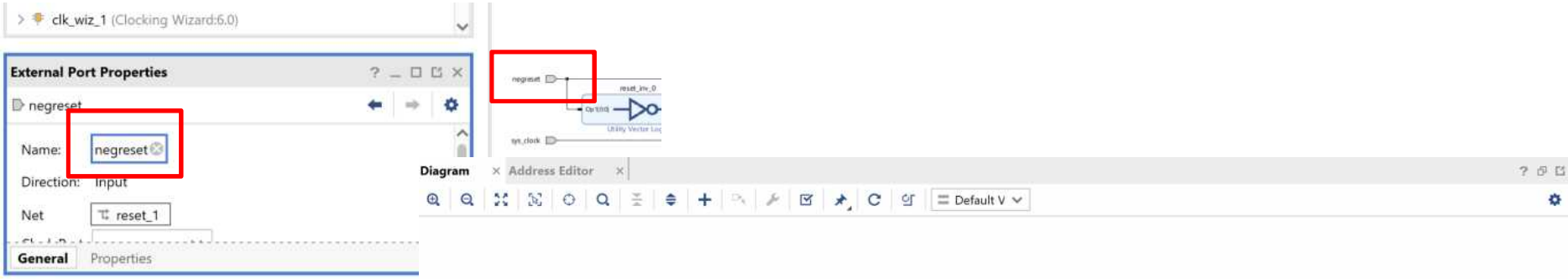


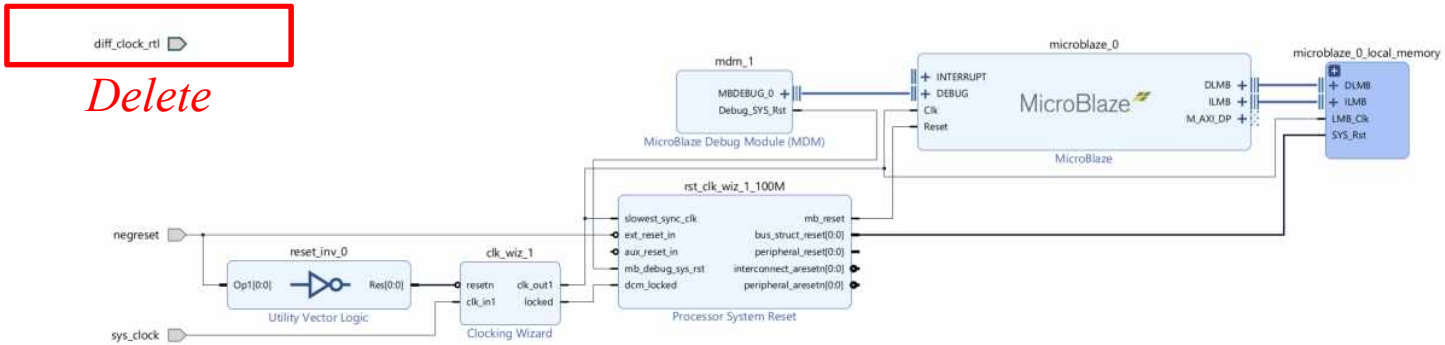


Hello World Implementation

➢ ***Hello World Implementation***

- 포트와 블록 정리 및 이름 변경
- ***reset_0 : negreset*** 으로 변경 (해당 핀 선택)➔ ***External Port Properties*** 창에서 ***Name : negreset*** 으로 변경)
- [***diff_clock_rtl 포트, reset_inv_0***] ***Block*** 제거 (선택 후 Delete 키로 제거)
- 배치 및 정리 ➔ ***Regenerate Layout*** 클릭

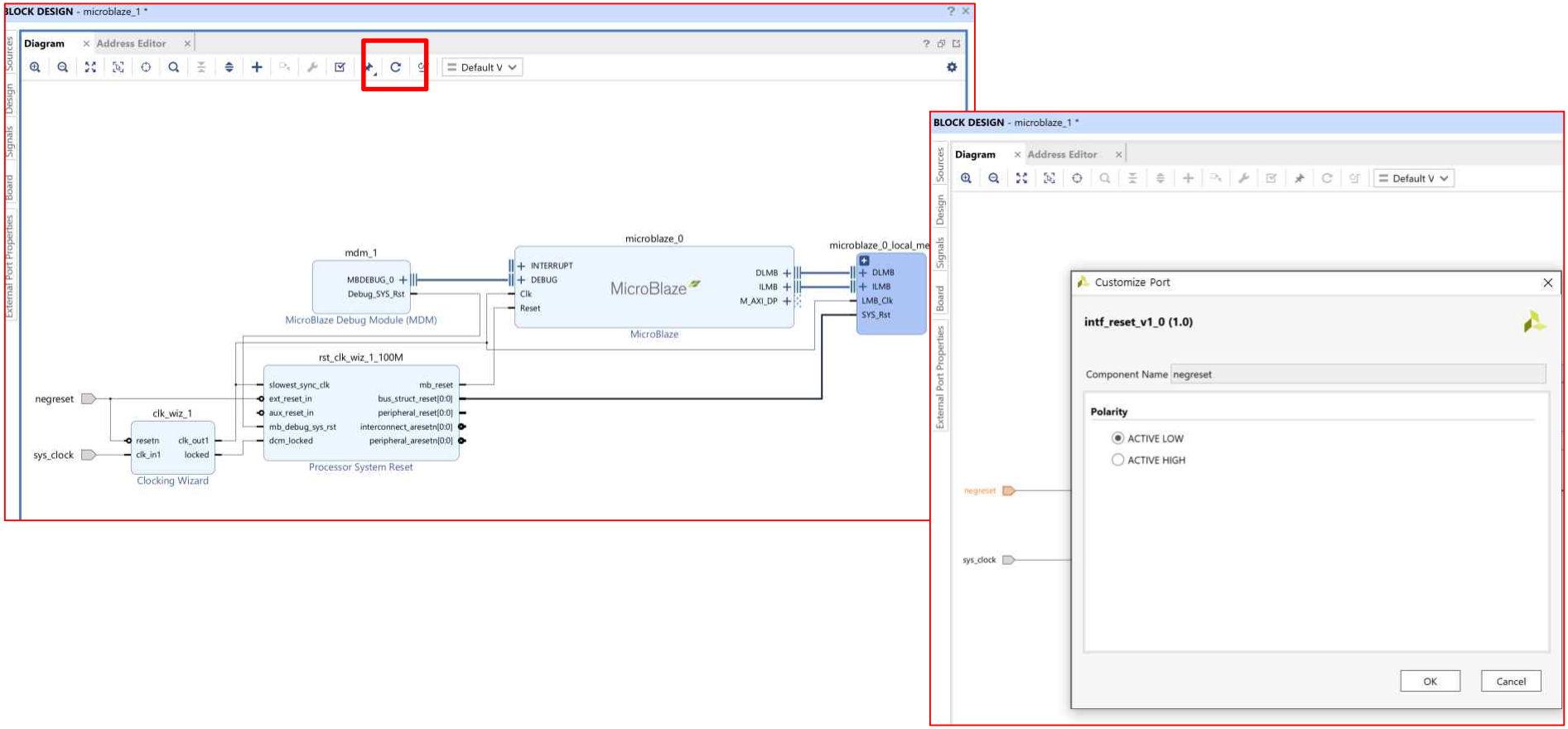




Hello World Implementation

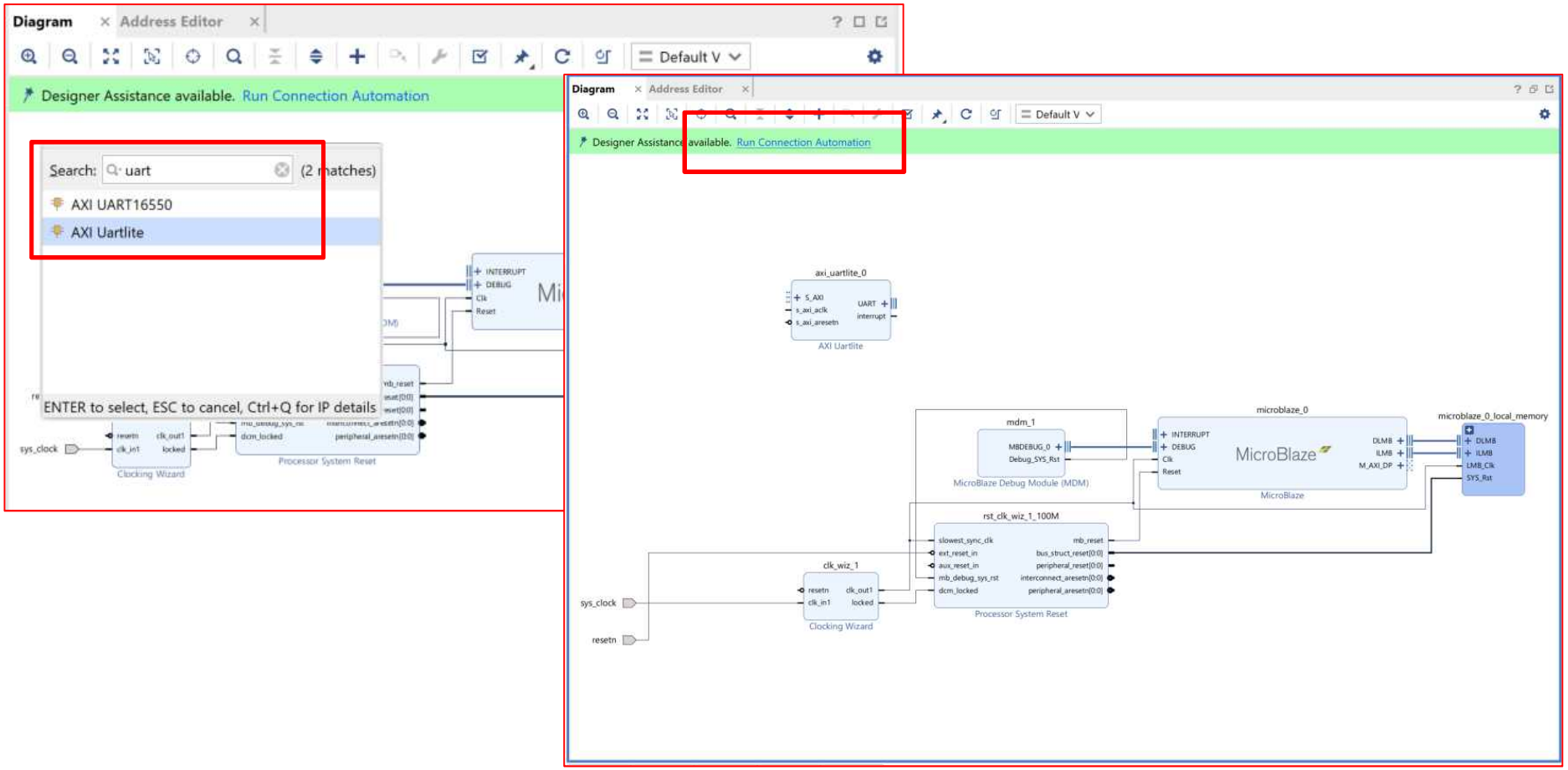
➢ ***Hello World Implementation***

- 포트와 블록 정리 및 이름 변경
- ***reset_0 : negreset*** 으로 변경 (해당 핀을 선택)➔ ***External Port Properties*** 창에서 ***Name : negreset*** 으로 변경)
- [***diff_clock_rtl*** 포트, ***reset_inv_0***] ***block*** 제거 (선택 후 ***Delete*** 키로 제거)
- 배치 및 정리 ➔ Regenerate Layout 클릭 ➔ [***negreset➔Active Low***변경]



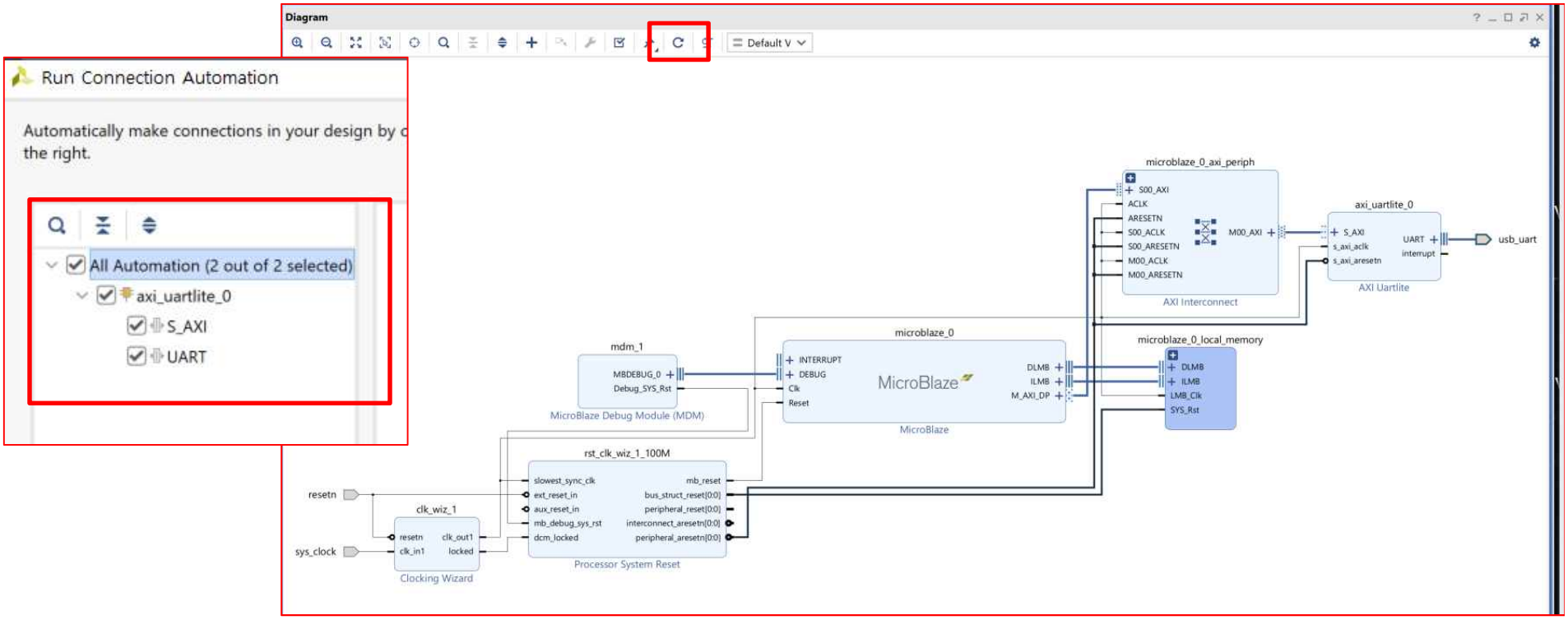
Hello World Implementation

- ***Hello World Implementation*** ➔ ***UART IP*** 추가
  - ***Add IP*** 아이콘 (+)을 클릭➔ ***UART Block***을 추가
  - ***UART*** 입력 ➔ [***AXI Uartlite***] → ***Double Click*** ➔ ***"Run Connection Automation"***을 클릭해서 연결
  - ***Regenerate Layout***



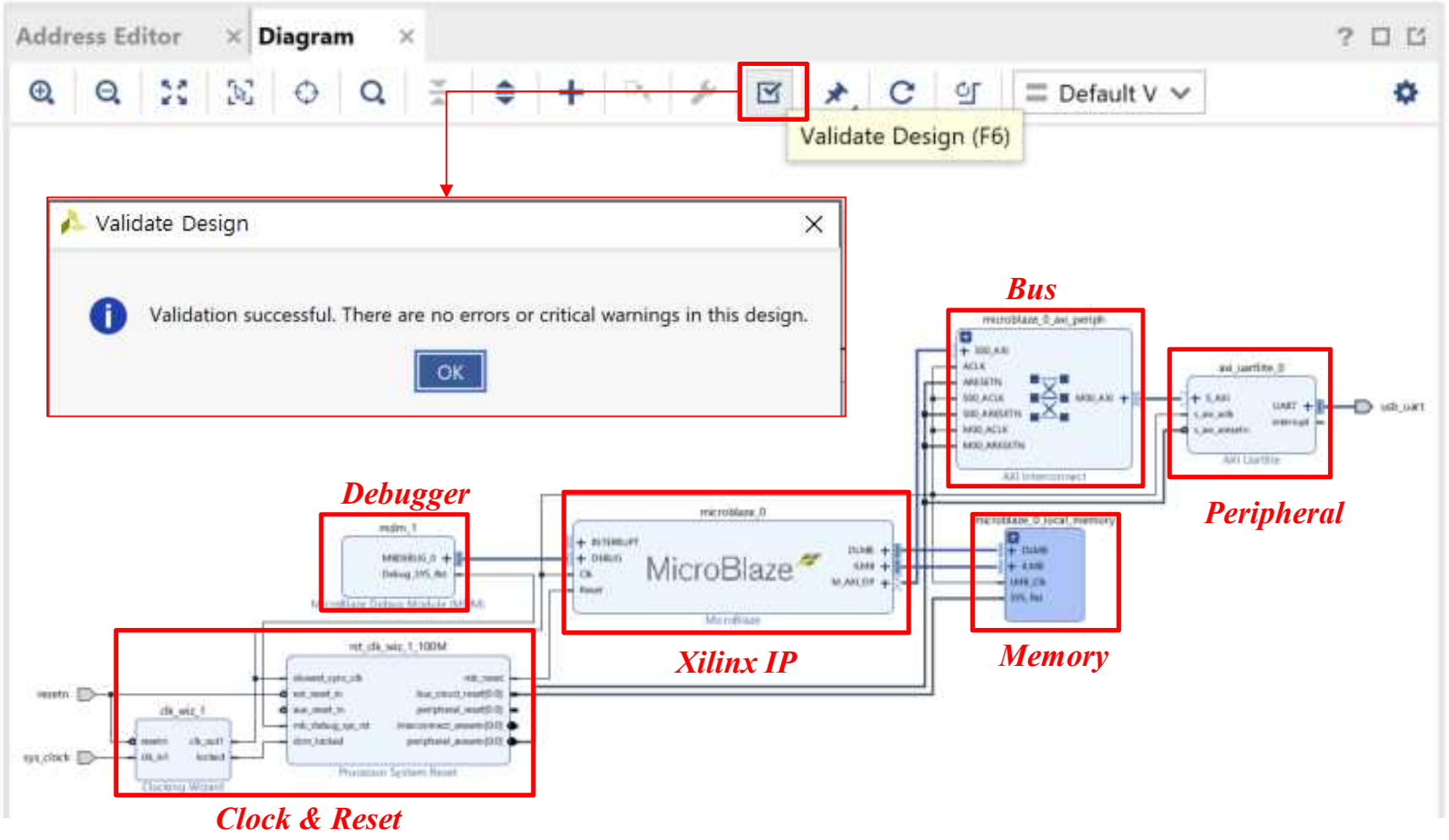
Hello World Implementation

- ***Hello World Implementation ➔ UART IP*** 추가
  - Add IP 아이콘 (+)을 클릭➔ UART Block을 추가
  - UART 입력 ➔ [***AXI Uartlite***] → Double Click ➔ "***Run Connection Automation***"을 클릭해서 연결



Hello World Implementation

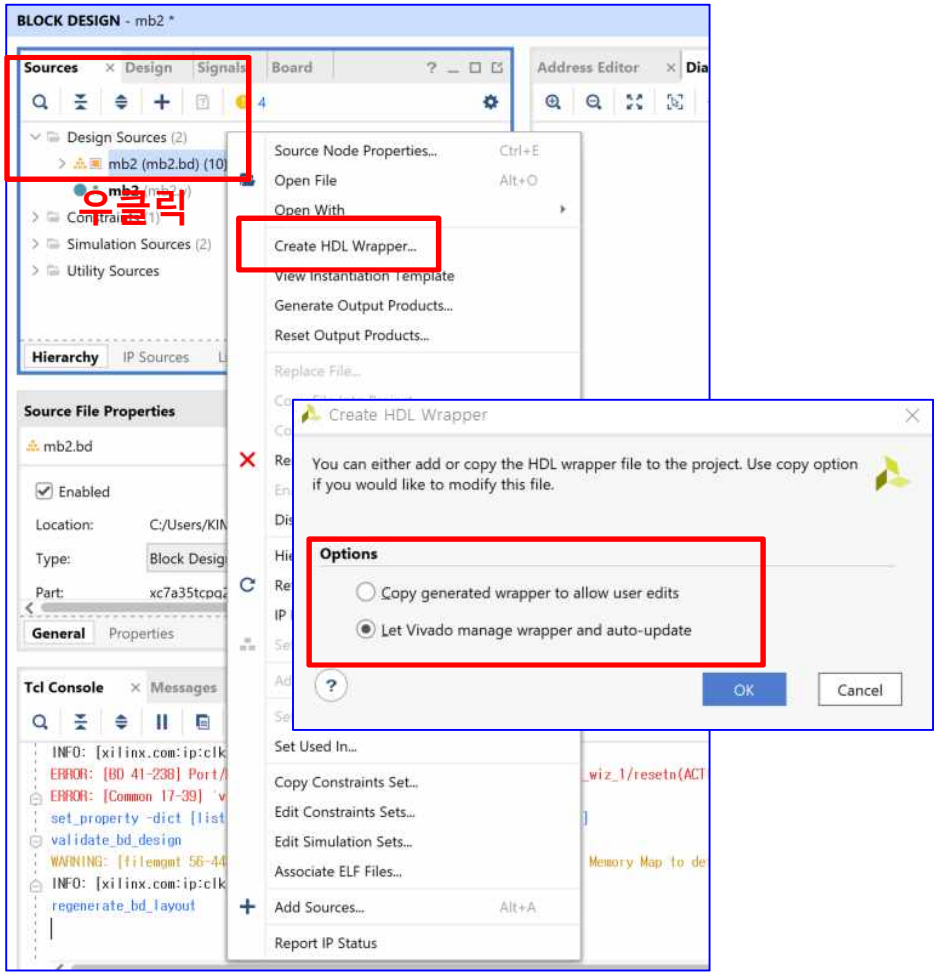
- ***Hello World Implementation***
  - IP Block ➔ 코드 변환
  - 유효성 검사 ➔ ***Validate Design*** 클릭 ➔ ***Validation Success***

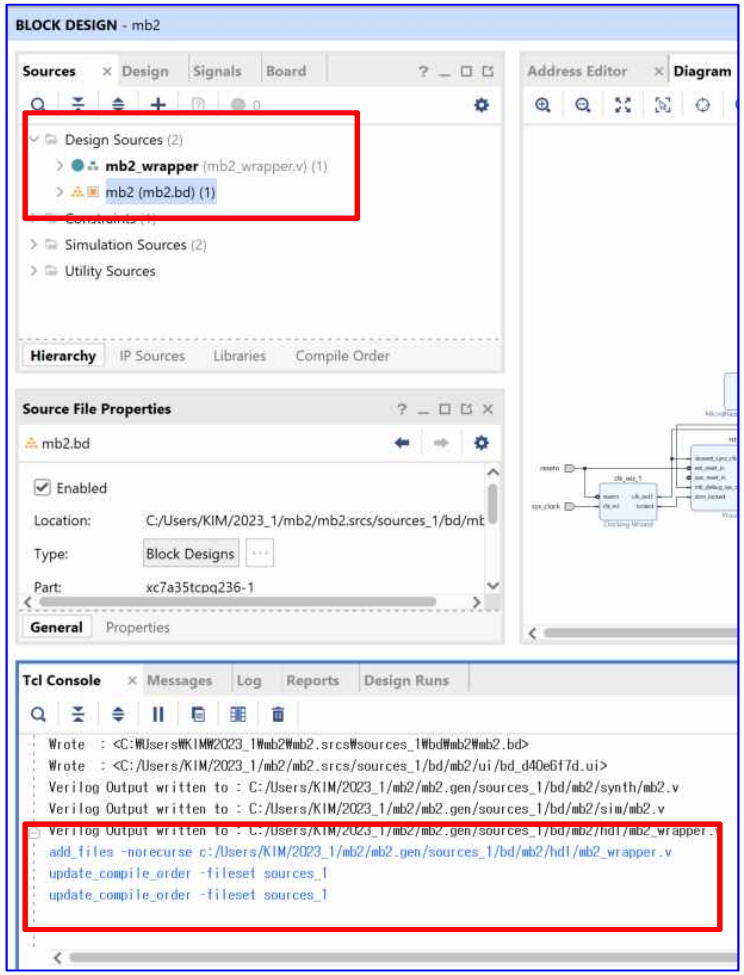


Hello World Implementation

➢ *Hello World Implementation*

- IP Block ➔ 코드 변환
- [*Sources*] ➔ [*Design sources*] ➔ [*microblaze*] 선택 ➔ 우클릭 ➔ [*Create HDL Wrapper...*] 클릭 ➔ OK





Hello World Implementation

- ***Hello World Implementation***
  - IP Block ➔ 코드 변환
  - ***[Sources] ➔ [Design sources] ➔ [microblaze]*** 선택 ➔ 우클릭 ➔ [***Create HDL Wrapper...***] 클릭 ➔ OK

Address Editor | Diagram | mb2.v | mb2_wrapper.v

c:/Users/KIM/2023_1/mb2/mb2.gen/sources_1/bd/mb2/hdl/mb2_wrapper.v

```
6  //Host        : DESKTOP-QVKKRPC running 64-bit major release  (build 9200
7  //Command     : generate_target mb2_wrapper.bd
8  //Design      : mb2_wrapper
9  //Purpose     : IP block netlist
10 //--------------------------------------------------------------------------
11 `timescale 1 ps / 1 ps
12
13 module mb2_wrapper
14    (resetn,
15     sys_clock,
16     usb_uart_rxd,
17     usb_uart_txd);
18   input resetn;
19   input sys_clock;
20   input usb_uart_rxd;
21   output usb_uart_txd;
22
23   wire resetn;
24   wire sys_clock;
25   wire usb_uart_rxd;
26   wire usb_uart_txd;
27
28   mb2 mb2_i
29        (.resetn(resetn),
30         .sys_clock(sys_clock),
31         .usb_uart_rxd(usb_uart_rxd),
32         .usb_uart_txd(usb_uart_txd));
33 endmodule
34
```

```
`timescale 1 ps / 1 ps
module mb2_wrapper
  (resetn,
  sys_clock,
  usb_UART_rxd,
  usb_UART_txd);
  input  resetn;
  input  sys_clock;
  input  usb_UART_rxd;
  output  usb_UART_txd;

  wire  resetn;
  wire  sys_clock;
  wire  usb_UART_rxd;
  wire  usb_UART_txd;

  mb2 mb2_i
      (.resetn(resetn),
       .sys_clock(sys_clock),
       .usb_UART_rxd(usb_UART_rxd),
       .usb_UART_txd(usb_UART_txd));
endmodule
```

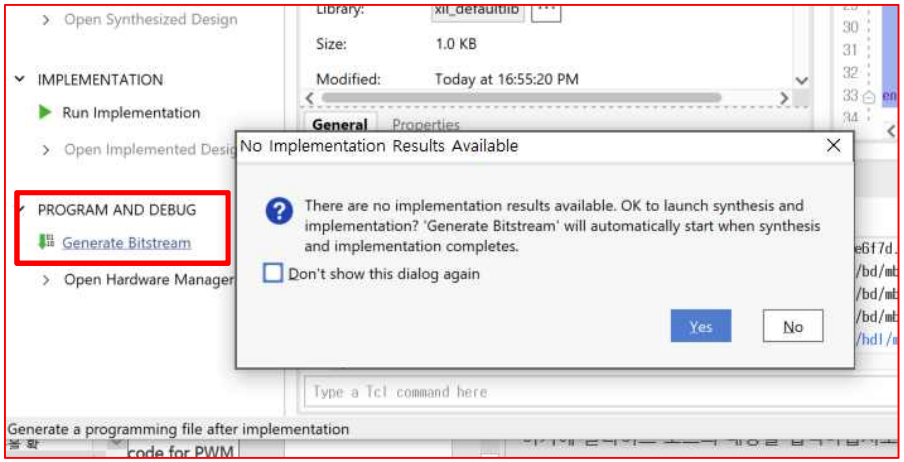
Hello World Implementation

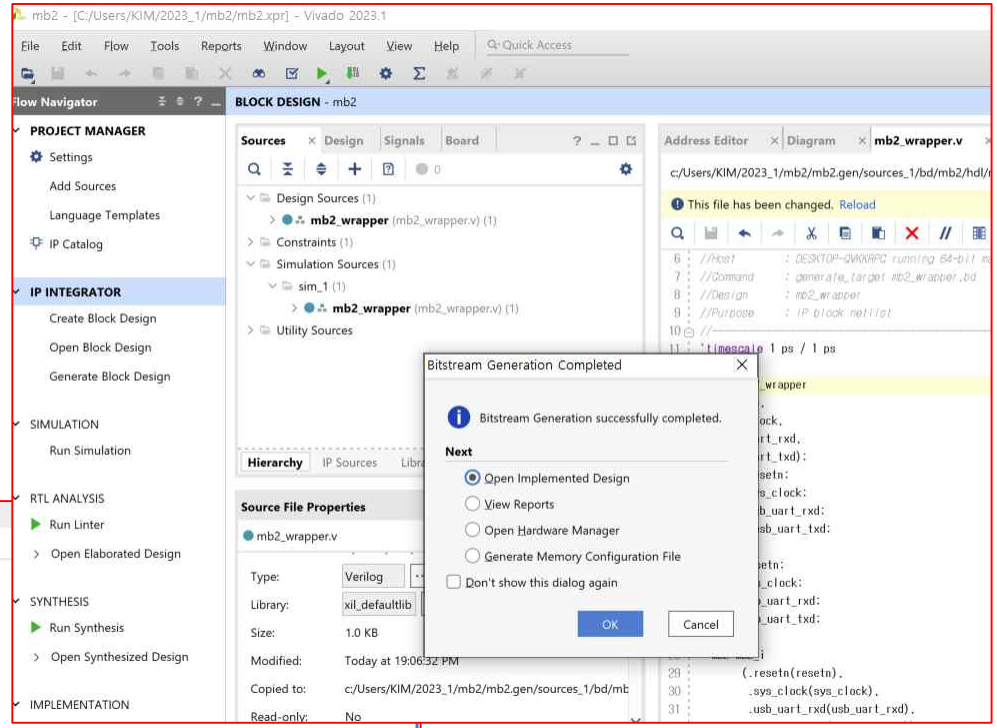
- *Hello World Implementation*
  - *Create Constraint File (*.XDC)*

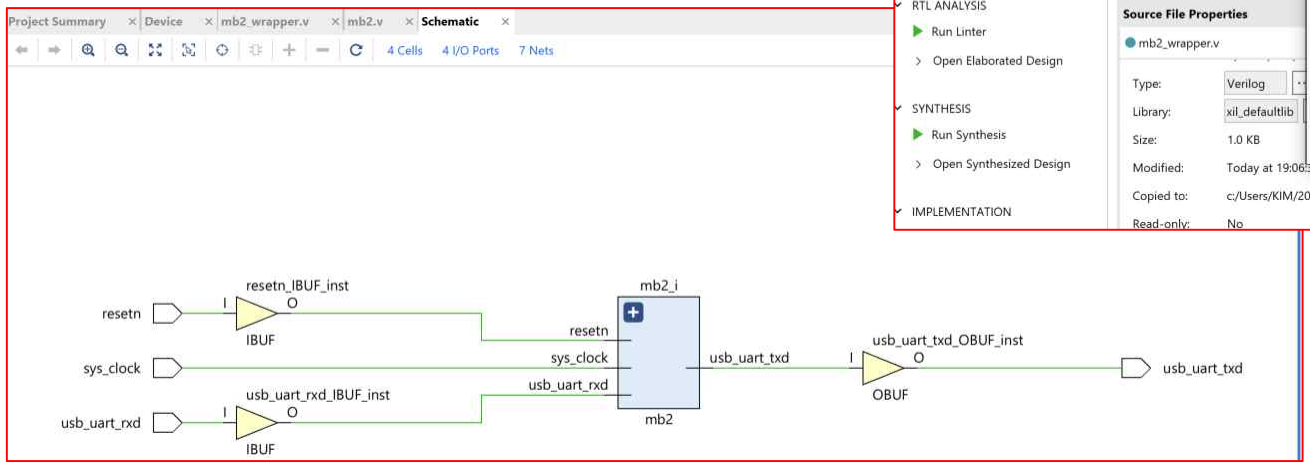
➔ 실습

Hello World Implementation

- ***Hello World Implementation***
  - ***IP ➔ Bitstream*** 생성
  - ***PROGRAM AND DEBUG ➔ Generate Bitstream*** [➔ 각 *IP* 코드 생성 , ***Synthesis*** 및 ***Implementation*** 진행됨]

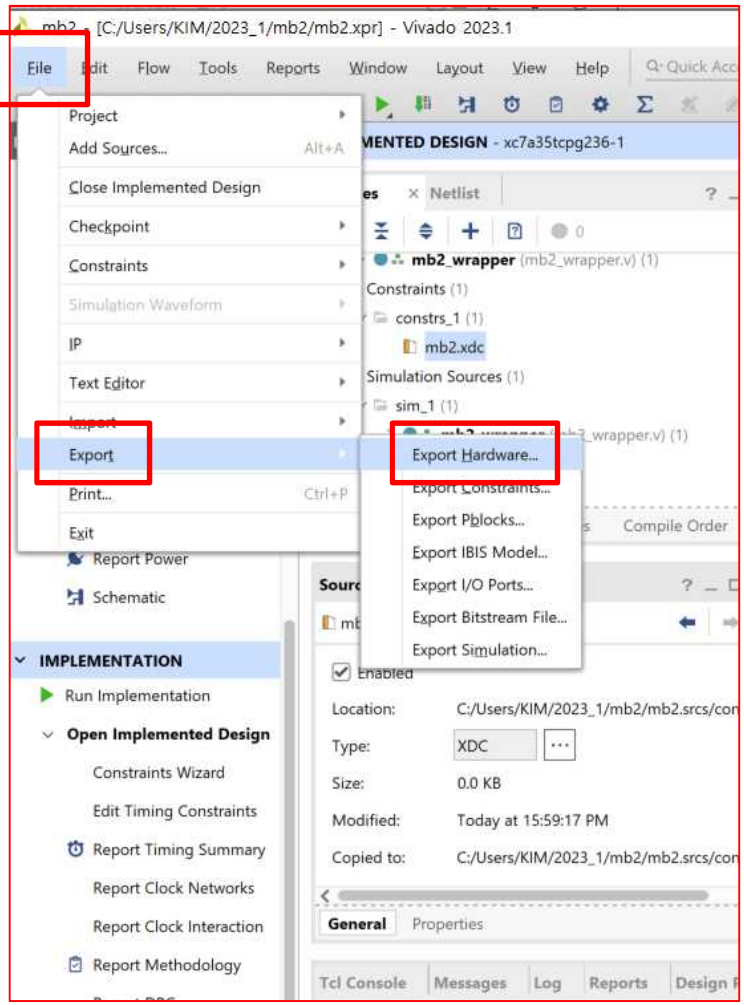


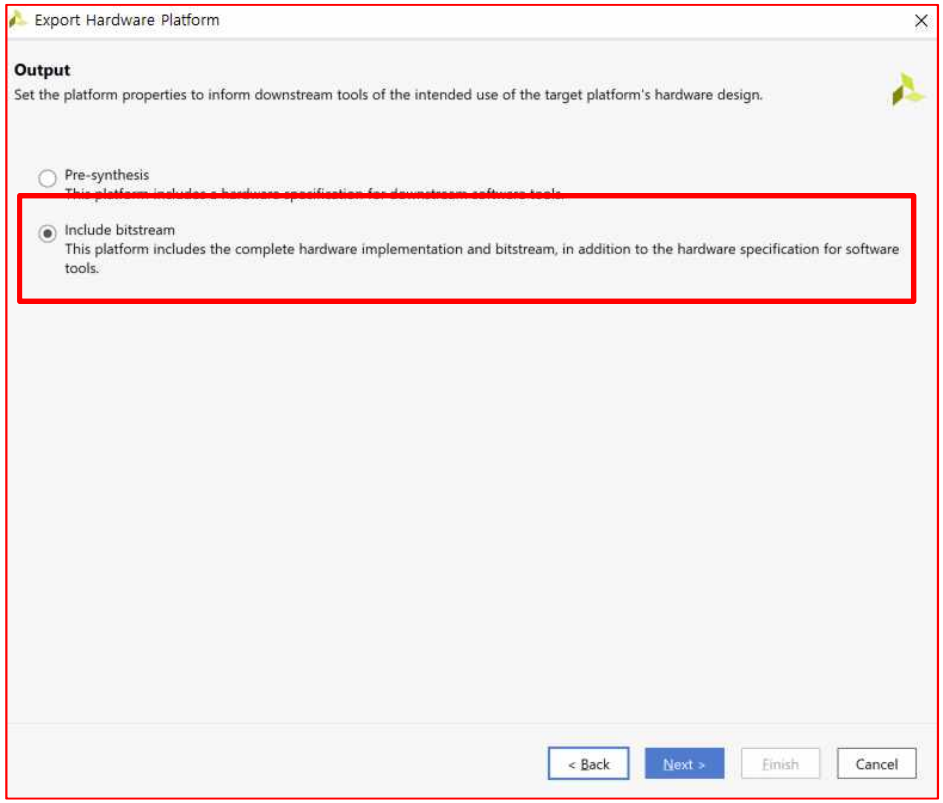




Hello World Implementation

- ***Hello World Implementation***
  - ***IP Block*** ➔ 코드 변환
  - ***[File]*** ➔ ***[Export]*** ➔ ***[Export Hardware..]*** 선택 ➔ ***Include bitstream*** ➔ ***Next***

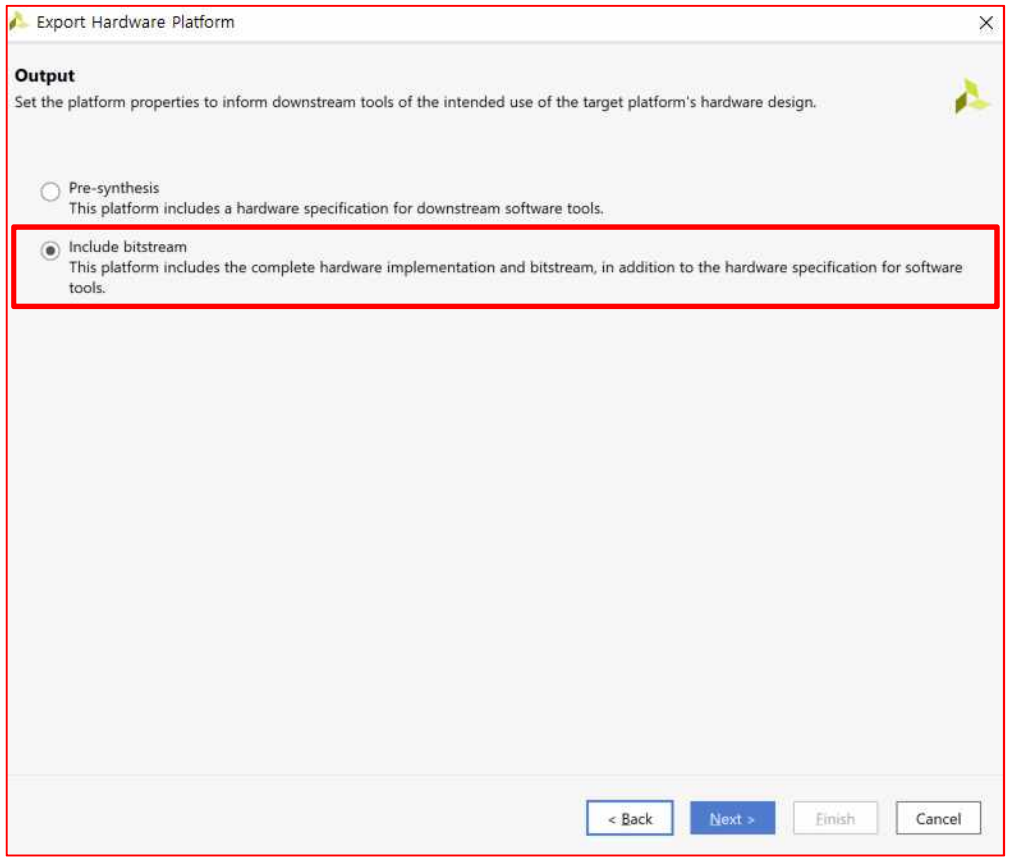


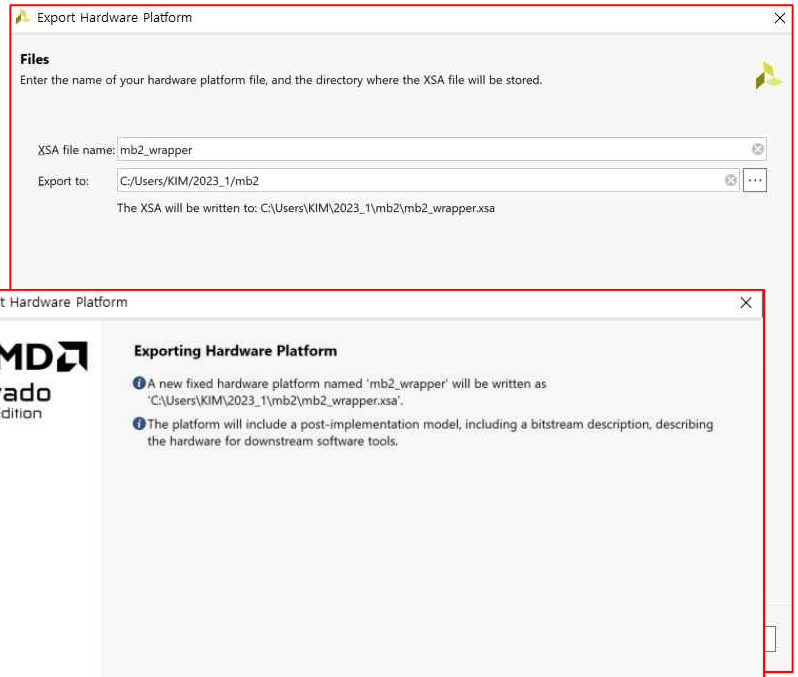


Hello World Implementation

➢ ***Hello World Implementation***

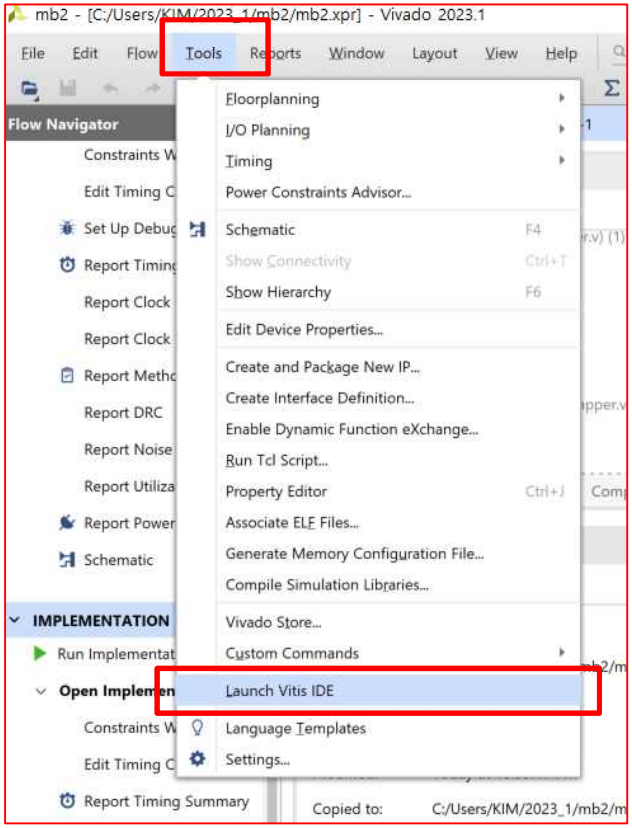
- ***Vitis*** 에서 사용하기 위하여 ***File*** ➔ ***Export - Export Hardware*** 실행 ➔ "***Include bitstream***" 선택➔ bitstream을 포함해서 ***Export***
- Bitstream을 포함해서 ***.xsa*** 파일 생성
- 소프트웨어 개발을 위한 ***XSA*** 파일 생성이 완료

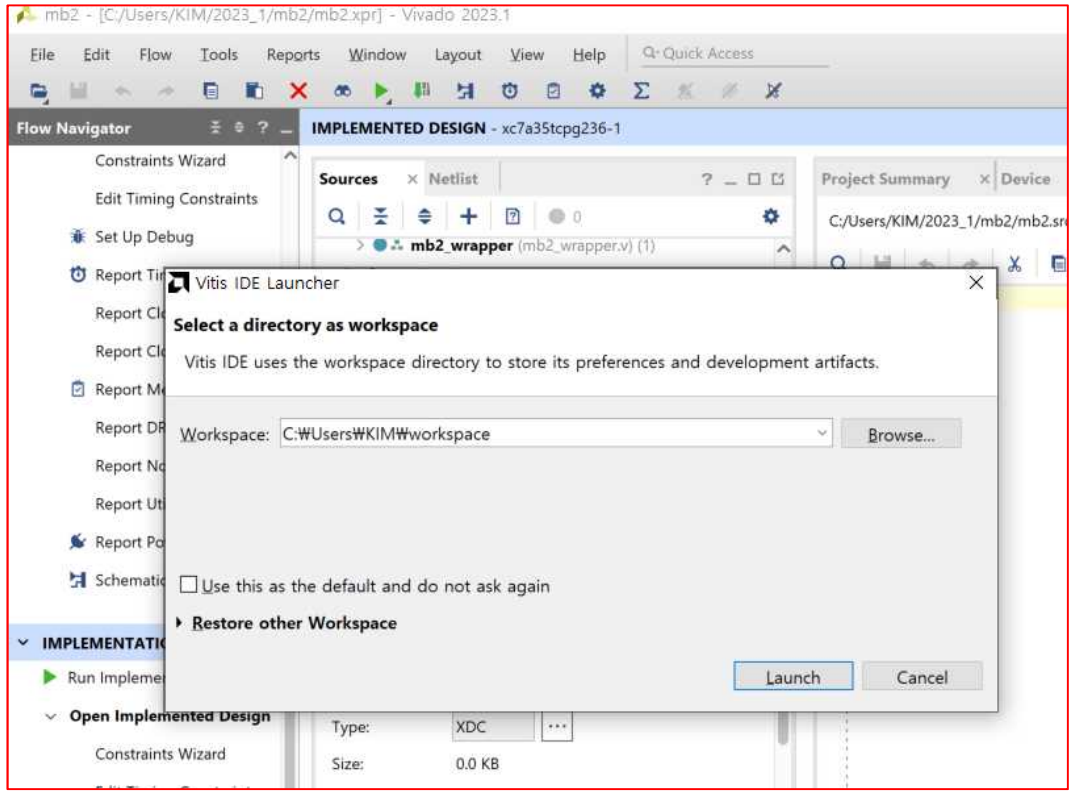




Hello World Implementation

- ***Hello World Implementation*** ➔ ***Vitis Application Implementation***
  - ***Application*** 구현 위한 ***MicroBlaze*** 생성이 완료
  - ***Application SW***을 구현하기 위하여 ***Vitis***를 실행 (방법1) : "***Vitis 2022.1***(or ***Vitis 2023.1)"*** 을 바로 실행
  - ***Application SW***을 구현하기 위하여 ***Vitis***를 실행 (방법2) : ***Vivado*** 에서 ***Tools*** ➔ ***Launch Vitis IDE*** 를 클릭
  - ***Vitis*** 실행 ➔ ***Workspace*** 폴더 지정 ➔➔ ***Vivado***에서 실행했던 폴더를 지정, HW와 SW를 같은 폴더에서 관리가 용이➔ ***Launch*** 버튼을 클릭해서 ***Vitis***를 시작





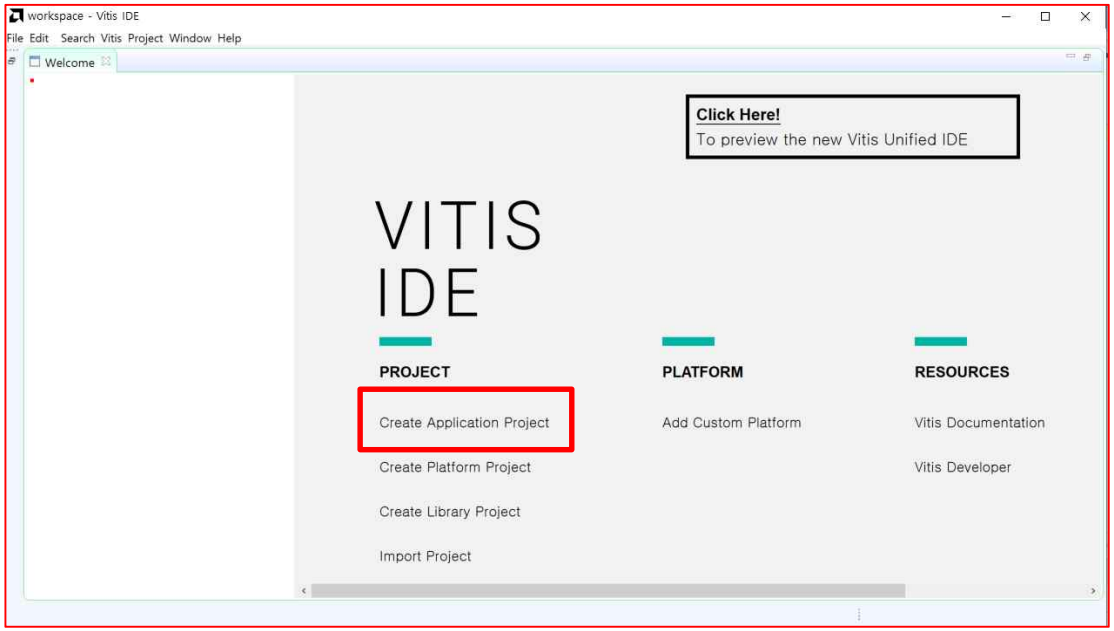
Hello World Implementation

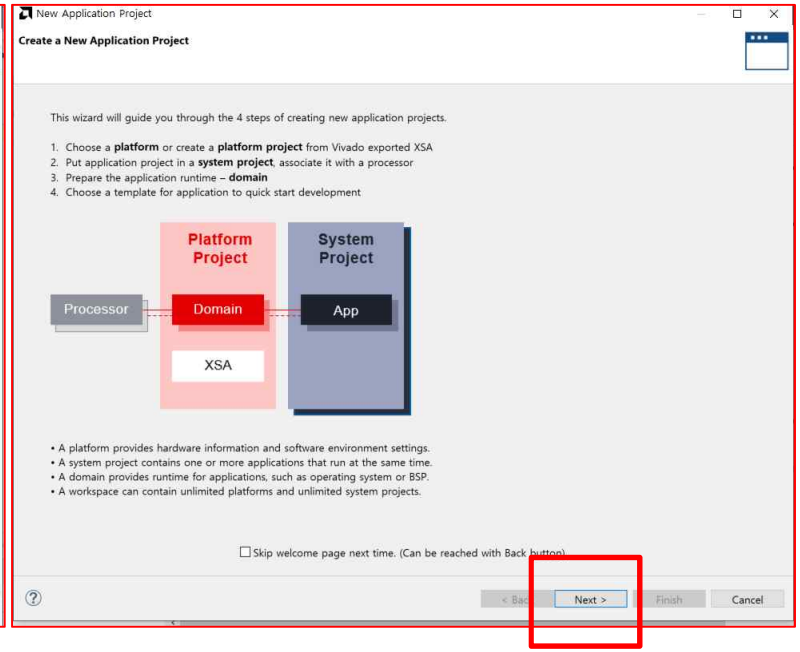
VIVADO(HW)

➔ ***VITIS IDE(AS, Application Software)***

Hello World Implementation

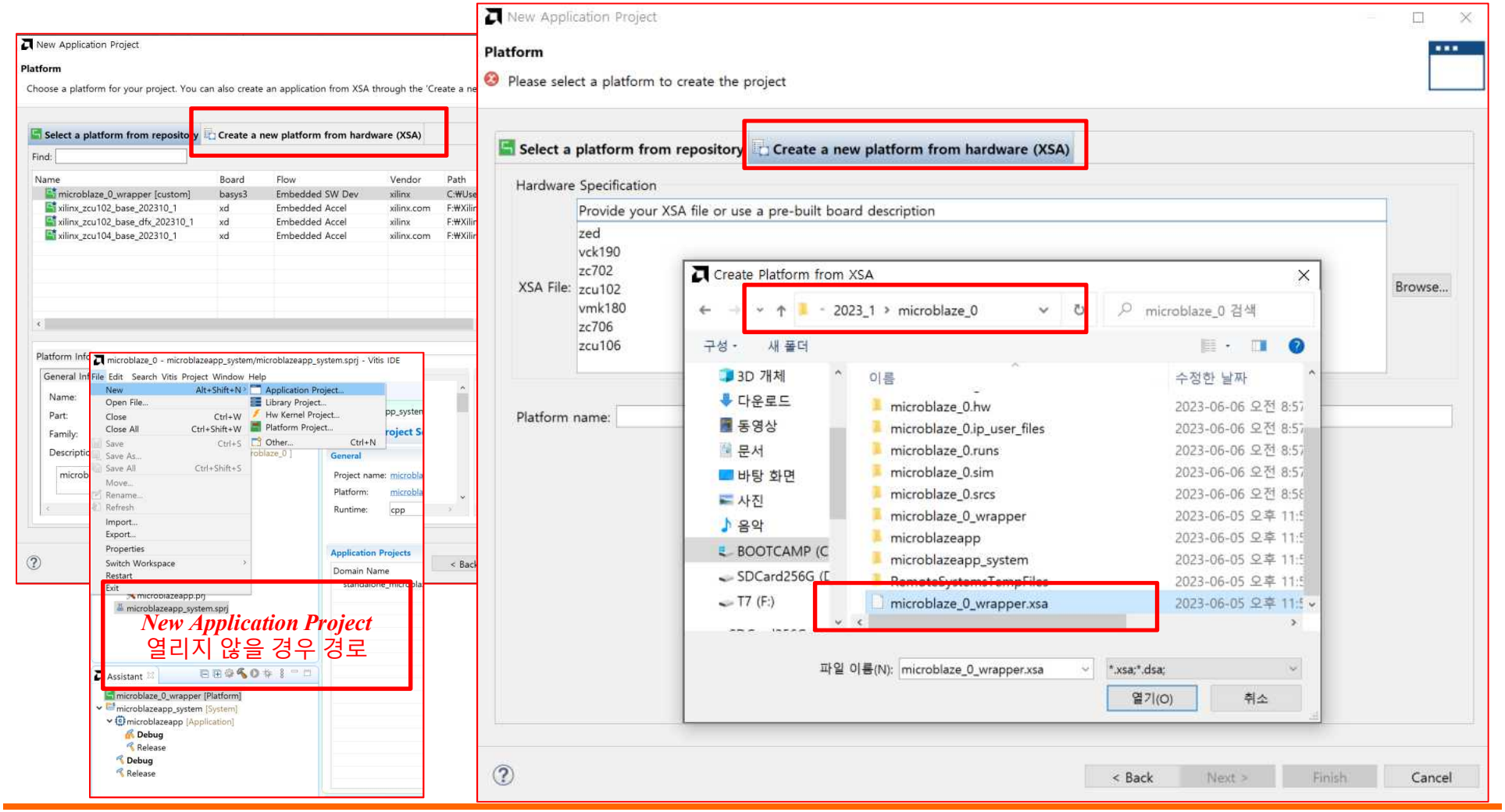
- ***Hello World Implementation***
  - ***Vitis*** 에서 2개의 프로젝트를 생성
  - ***Vivado*** 에서 구현한 ***HW(xsa*** 파일***)***를 설정하는 ***Platform Project***
  - 두번째는 ***Application SW***를 구현하는 ***Application Project***
  - ***Vitis***가 ***2022.1*** 버전으로 업데이트 되면서 ***Application Project*** 내에서 ***Platform Project***를 생성가능
  - "***Create Application Project***"를 클릭





Hello World Implementation

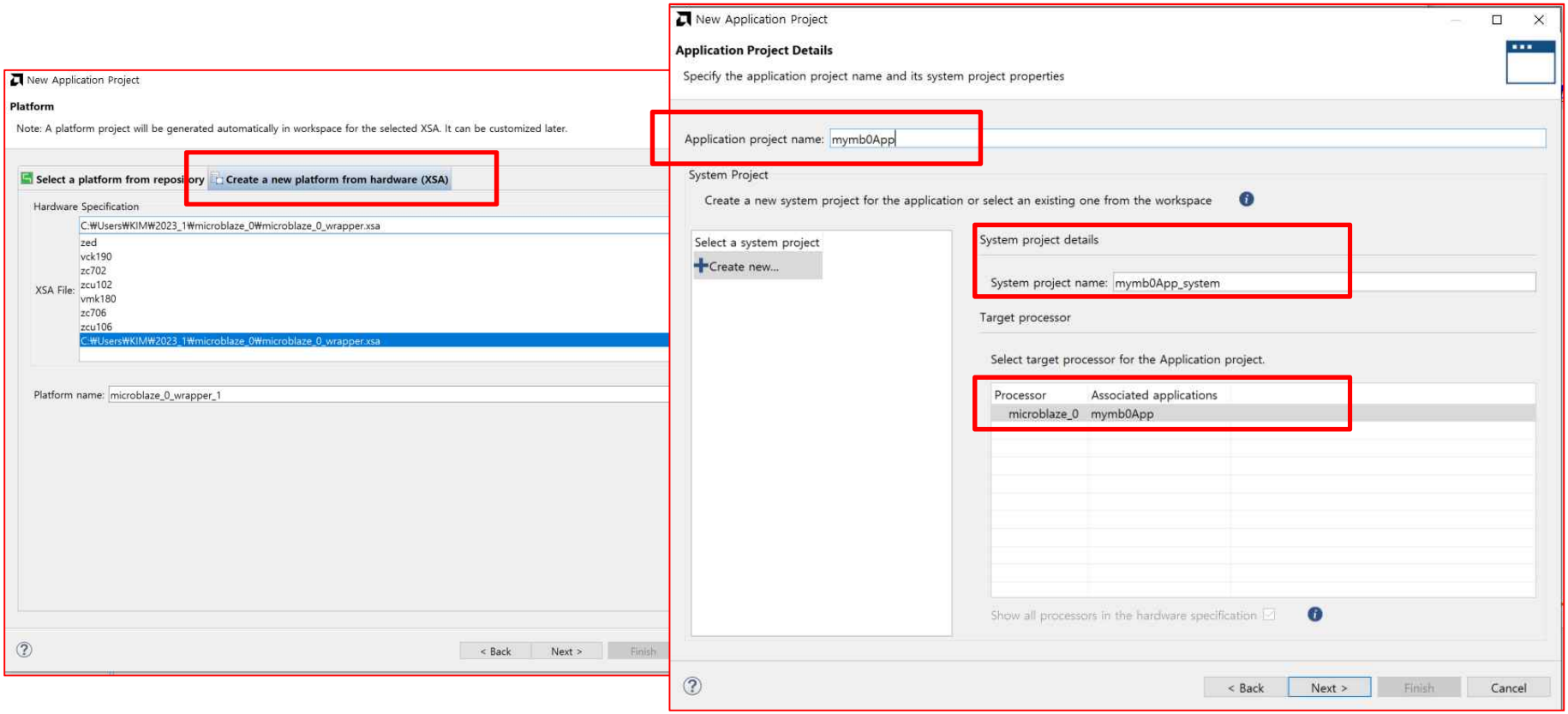
- ***Hello World Implementation***
  - ***New Application Project*** ➔ "***Create a new platform from hardware (XSA)*** ➔ ***Platform Project*** 생성
  - ****_wrapper.xsa*** 파일 열기



Hello World Implementation

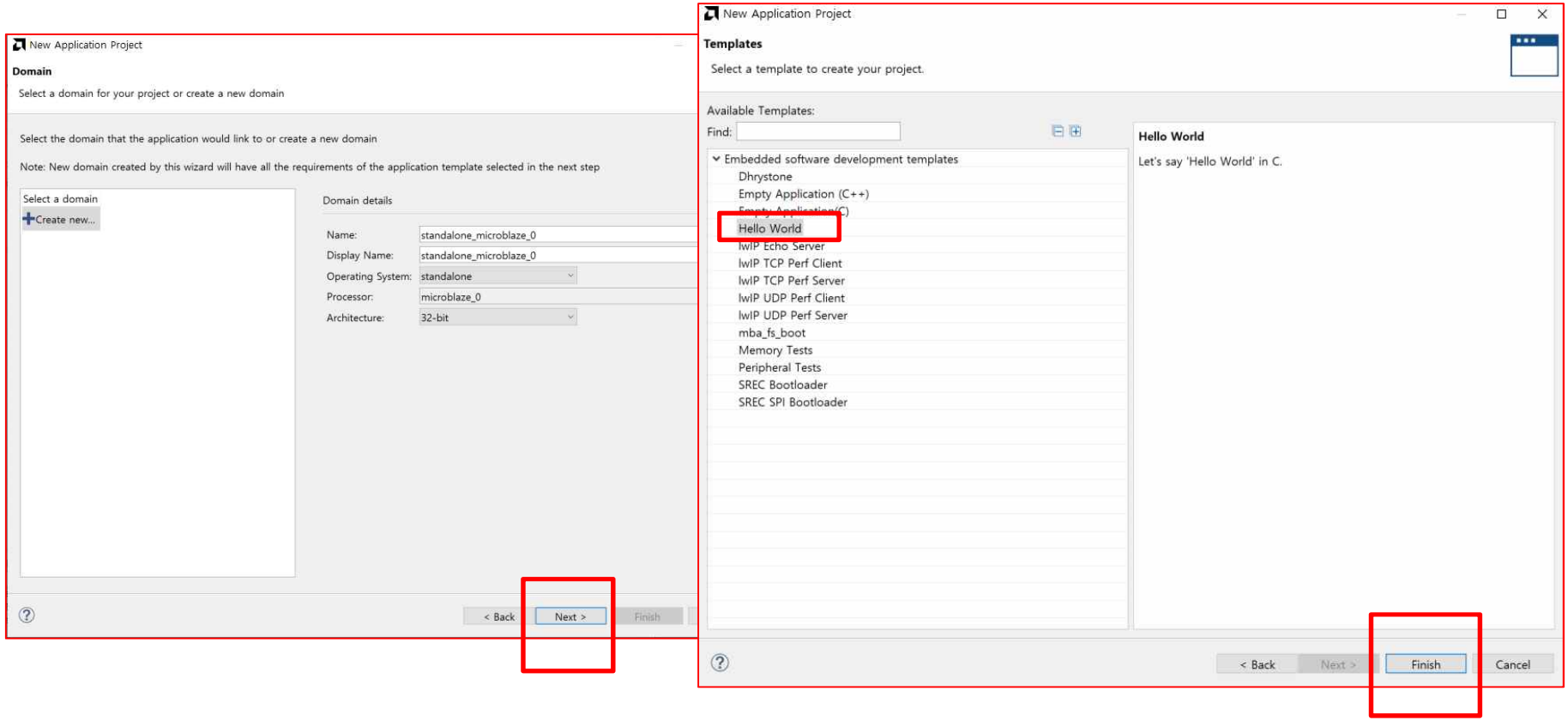
➢ ***Hello World Implementation***

- ***New Application Project*** ➔ ***"Create a new platform from hardware (XSA)*** ➔ ***Platform Project*** 생성
- [****_wrapper.xsa***] 파일 열기 ➔ Next
- ***Application project Name*** 설정 : ***(ex) mymb0App [System project name : mymb0App_system]*** ➔ ***Next***



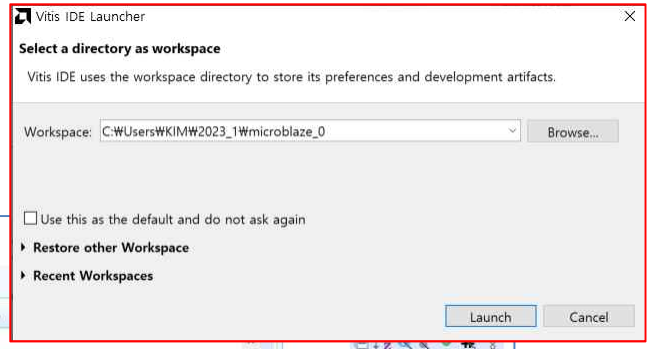
Hello World Implementation

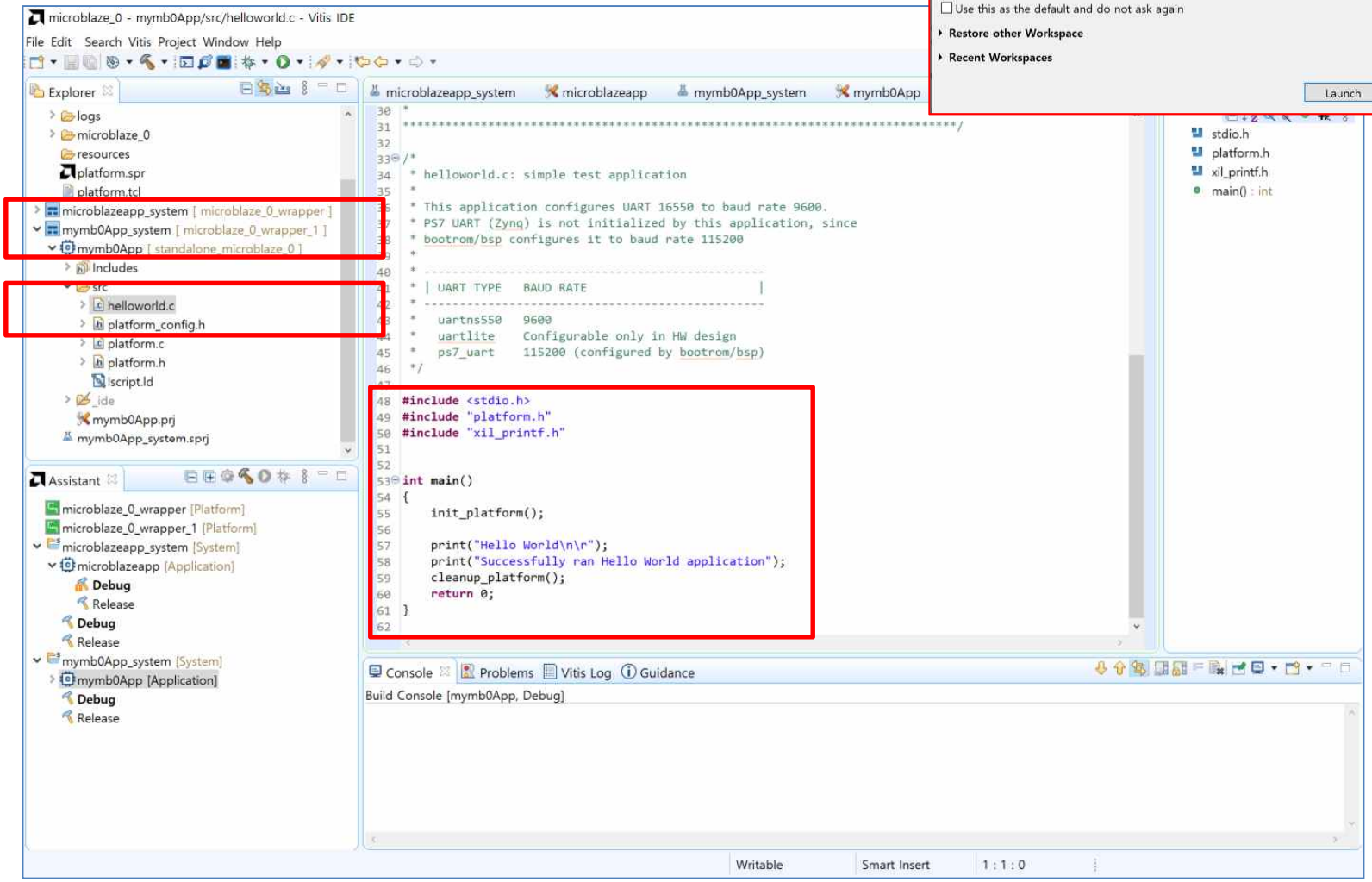
- ***Hello World Implementation***
  - ***New Application Project ➔ "Create a new platform from hardware (XSA) ➔ Platform Project*** 생성
  - [****_wrapper.xsa***] 파일 열기 ➔ Next
  - ***Application project Name*** 설정 ➔ ***Next***
  - ***Template ➔ Available Templates ➔ "Hello World" ➔ Finish***



Hello World Implementation

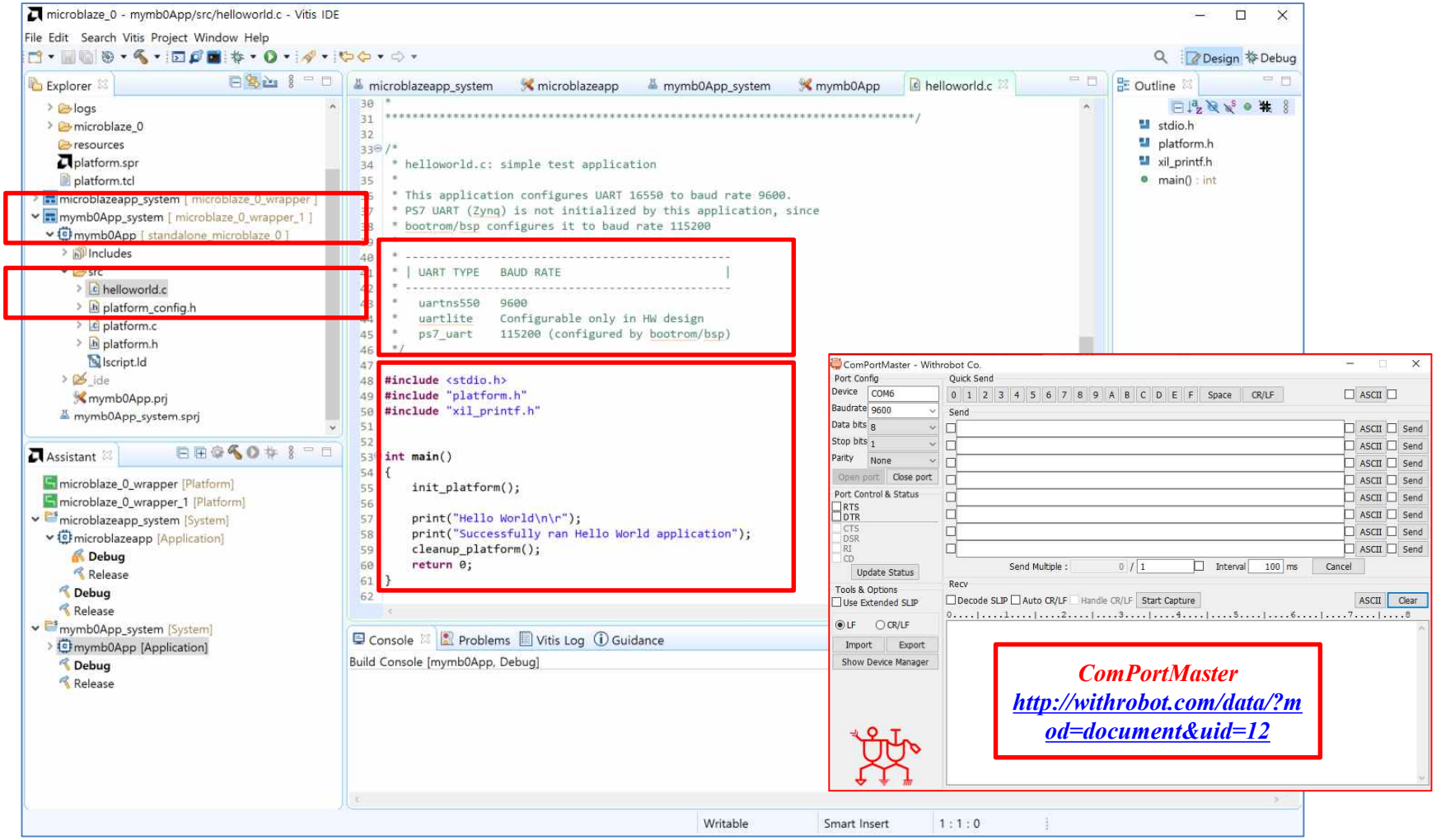
- ***Hello World Implementation***
  - ***helloworld.c*** ➔ 클릭





Hello World Implementation

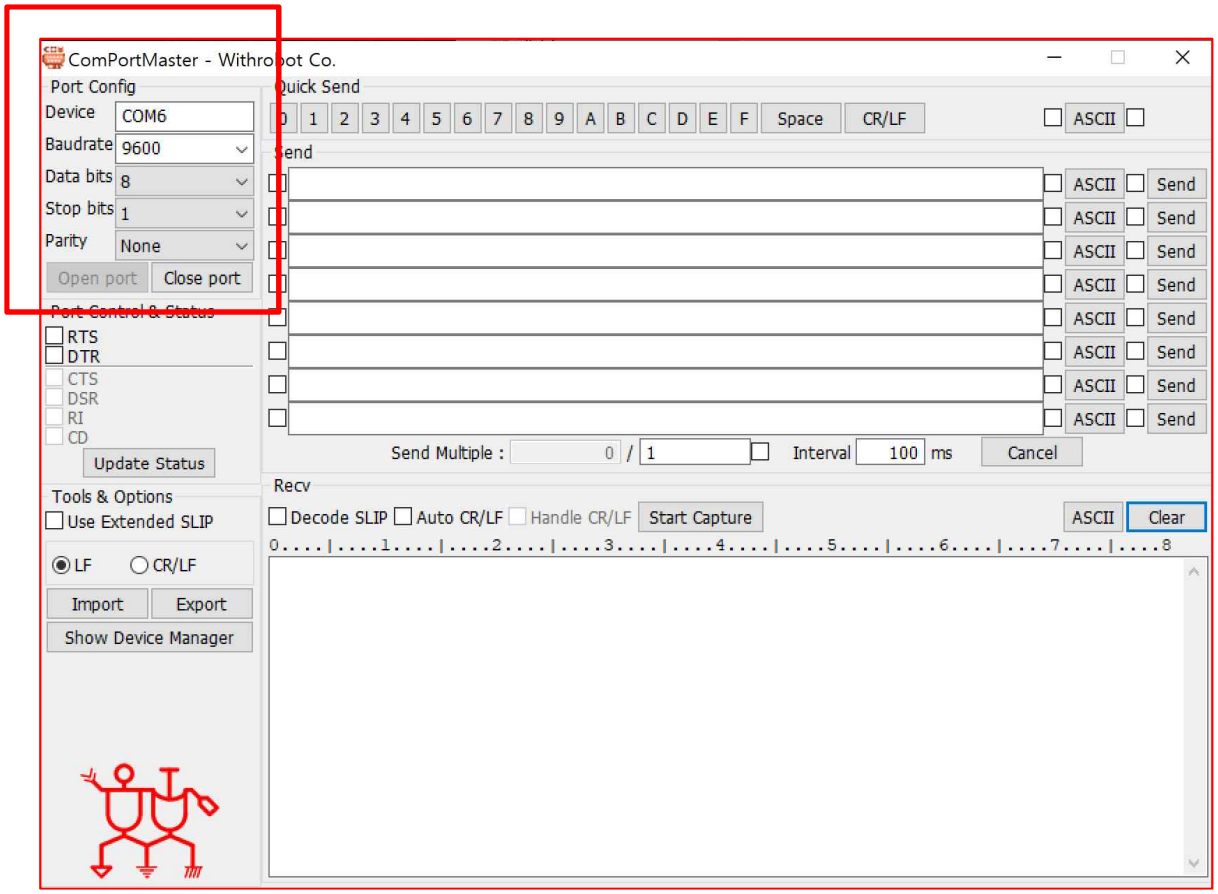
- ***Hello World Implementation***
  - ***helloworld.c*** ➔ 클릭
  - ***UART TYPE : UARTns550 ➔ BAUD RATE : 9,600***



ComPortMaster
http://withrobot.com/data/?mod=document&uid=12

Hello World Implementation

- ***Hello World Implementation***
  - ***ComPortMaster*** :직렬(시리얼)통신 터미널 프로그램 [***http://withrobot.com/data/?mod=document&uid=12***]

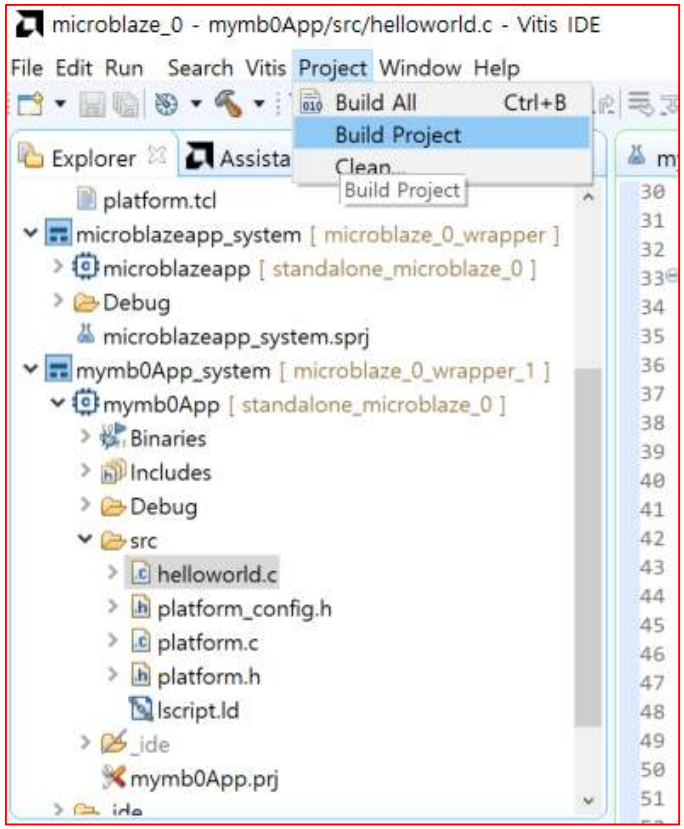


Hello World Implementation

- ***Hello World Implementation***
  - ***Project ➔ Build Project ➔ Program Device***
  - *Vitis ➔ Program Device ➔ Browse ➔*

    C:\Users\KIM\2023_1\microblaze_0\mymb0App\Debug\mymb0App.elf ➔ 선택 , 열기 ➔ *Program*

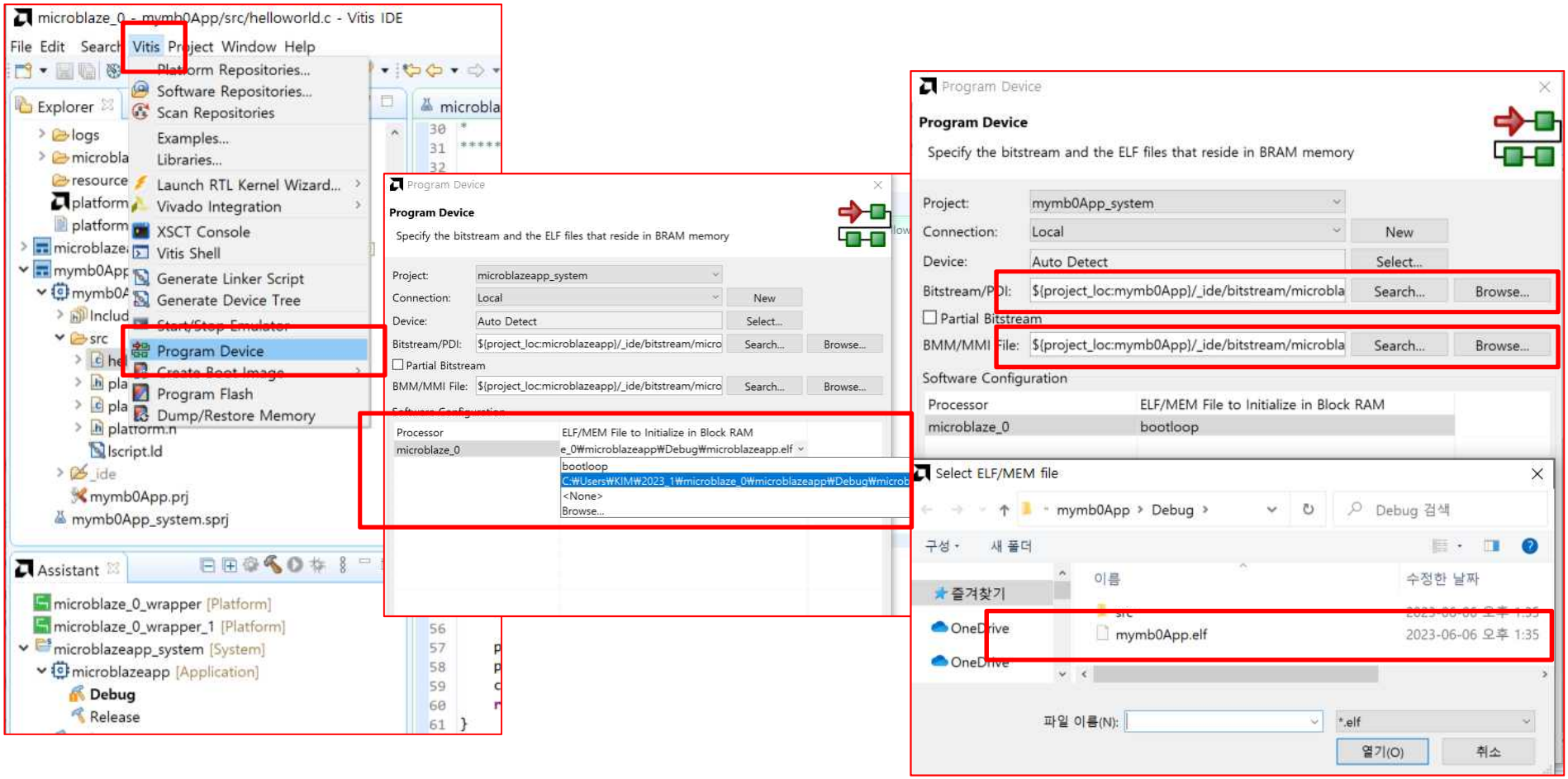
Project ➔ Build Project



Hello World Implementation

- ***Hello World Implementation***
  - ***Vitis ➔ Program Device ➔ Browse ➔***

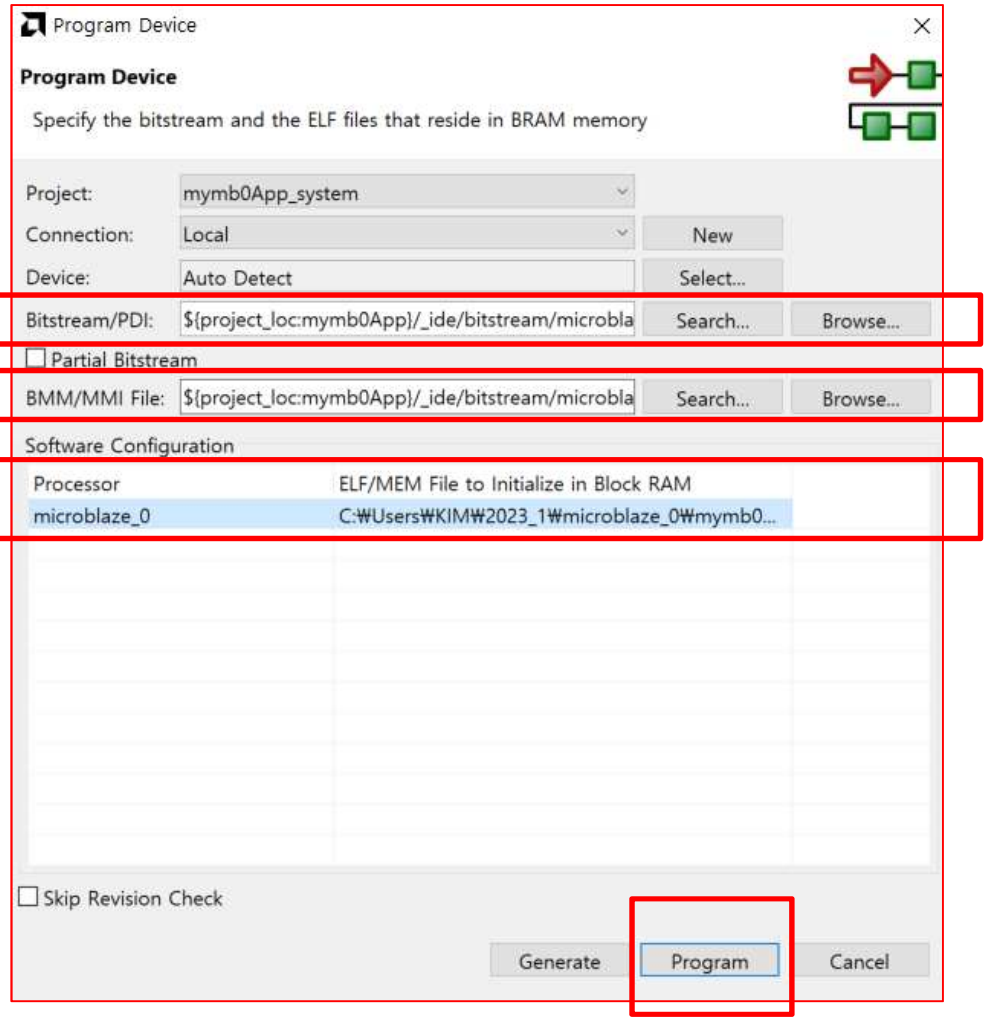
    C:\Users\KIM\2023_1\microblaze_0\mymb0App\Debug\mymb0App.elf ➔ 선택 , 열기 ➔ ***Program***



Hello World Implementation

- ***Hello World Implementation***
  - ***Vitis ➔ Program Device ➔ Browse ➔***

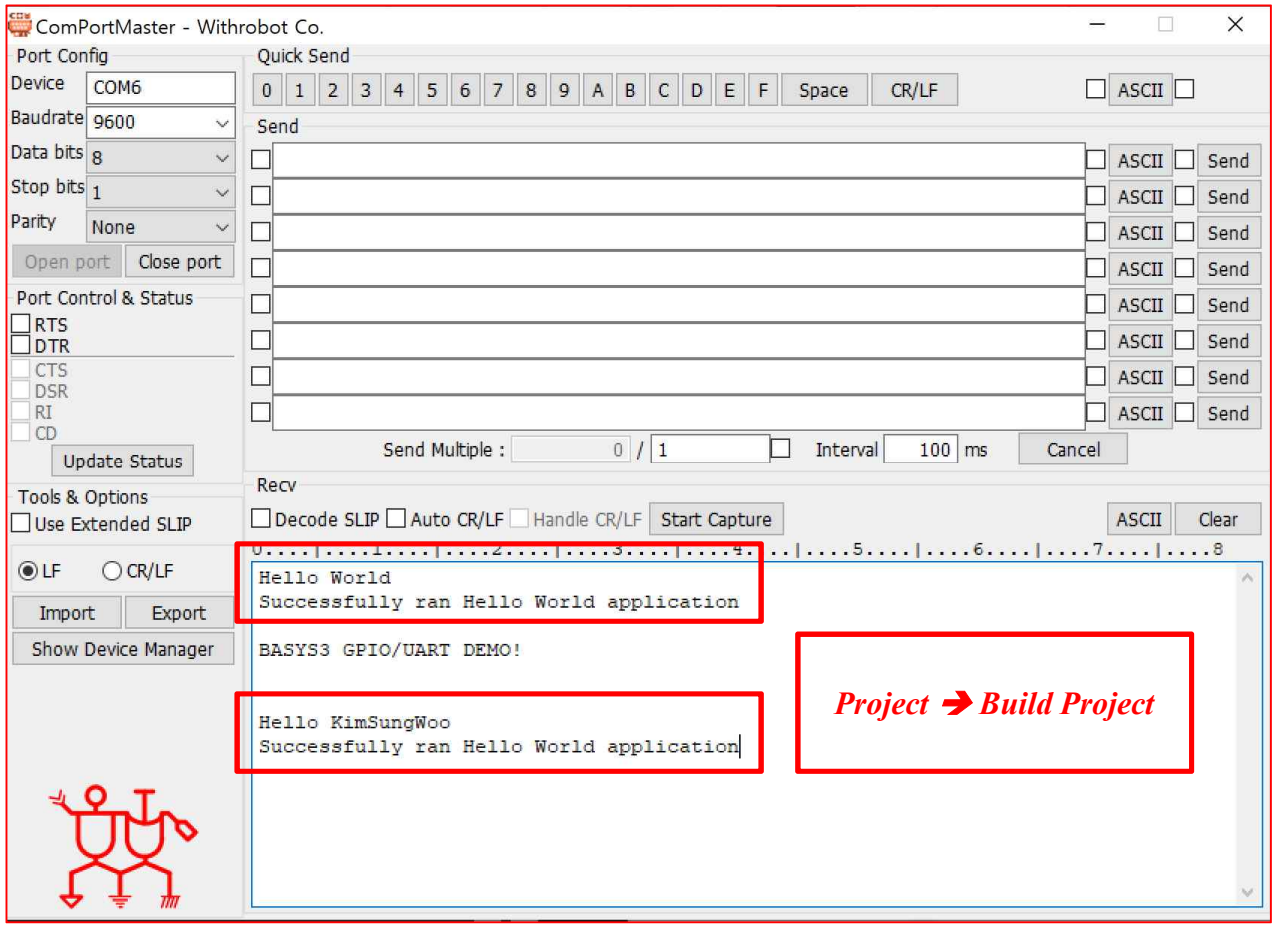
    C:\Users\KIM\2023_1\microblaze_0\mymb0App\Debug\mymb0App.elf ➔ 선택 , 열기 ➔ ***Program***

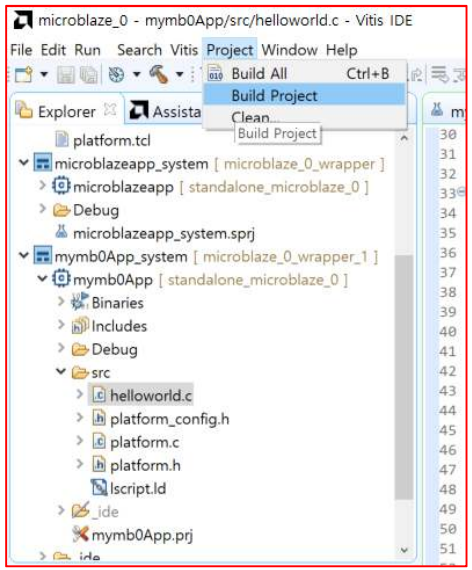


Hello World Implementation

- *Hello World Implementation*
  - *Vitis ➔ Program Device ➔ Browse ➔*

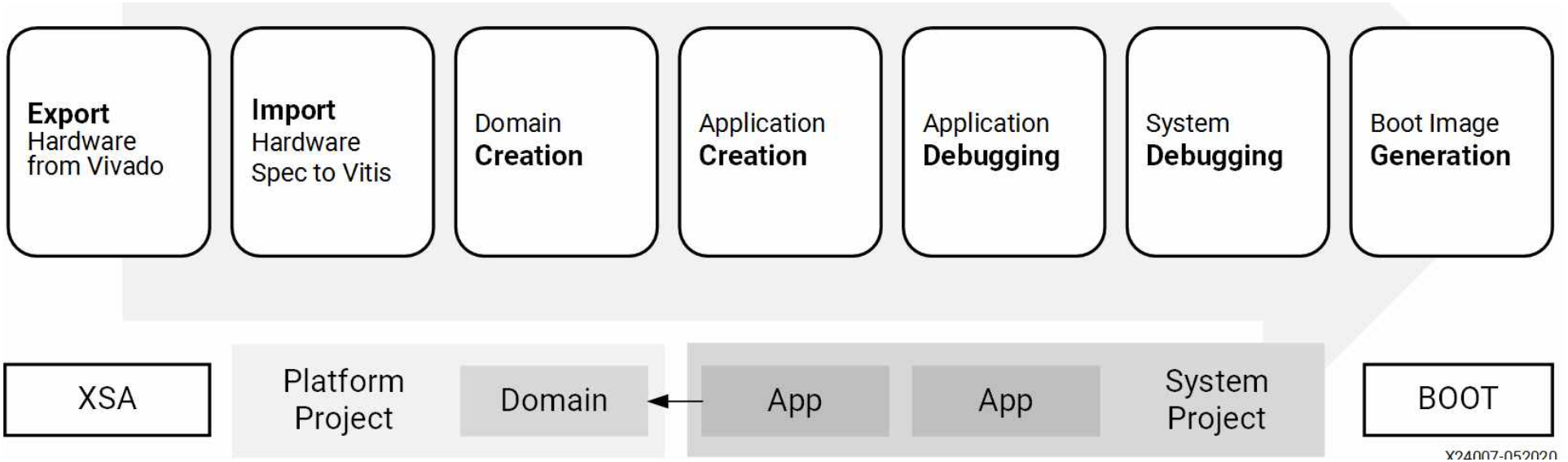
    C:\Users\KIM\2023_1\microblaze_0\mymb0App\Debug\mymb0App.elf ➔ 선택 , 열기 *➔ Program*
  - *Project ➔ Build Project ➔ Program Device ➔ Hello World Implementation Result Check using UART*



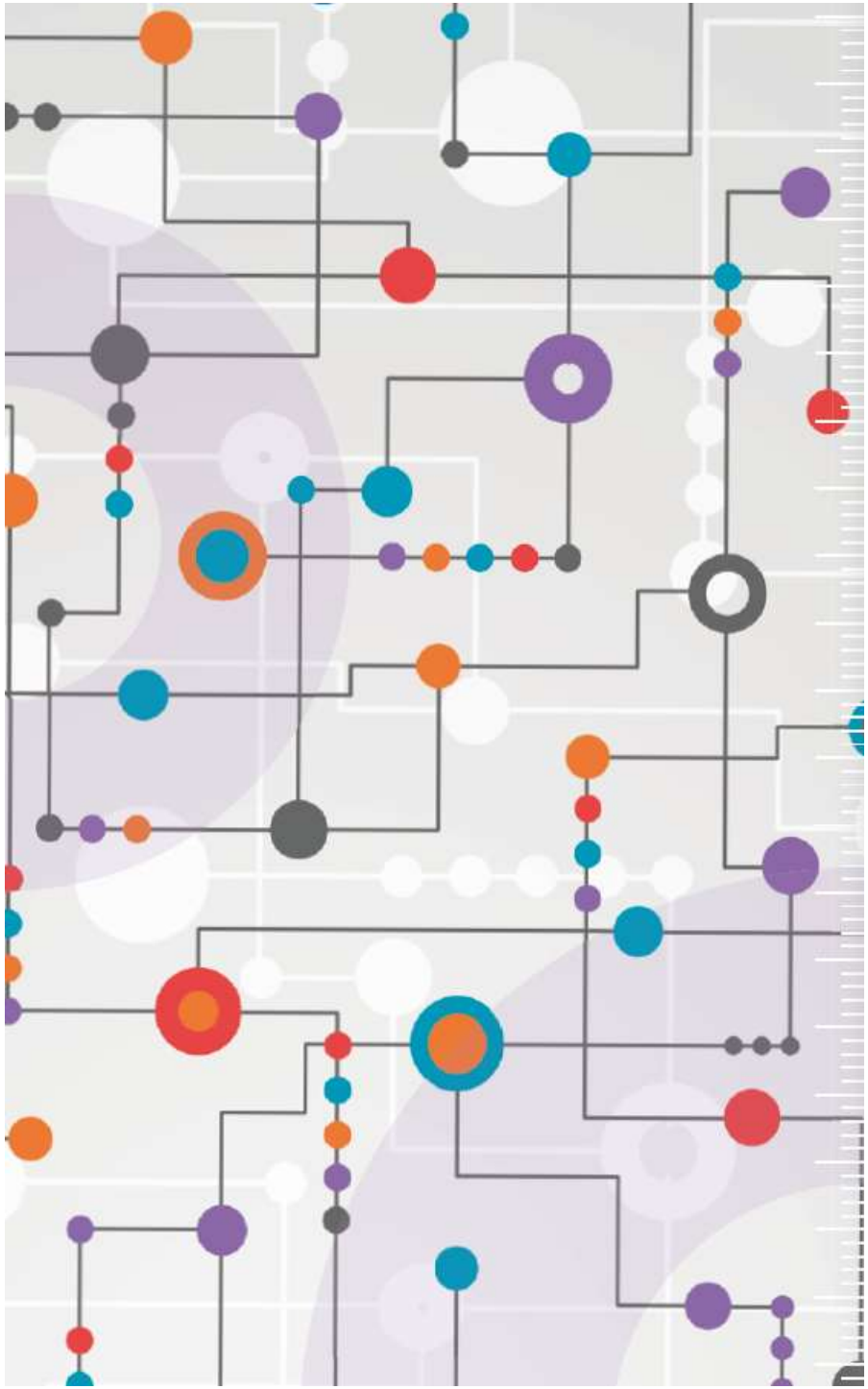


Hello World Implementation

- ***VITIS*** 소프트웨어 구현 [***https://docs.xilinx.com/r/en-US/ug1400-vitis-embedded/Getting-Started-with-Vitis***]
  - ***VIVADO***를 이용해서 ***xsa*** 확장자인 하드웨어(HW)를 만들고 이 파일을 바탕으로 ***VITIS***에서 사용 할 수 있는 도메인***(Domain)***을 생성 ➔도메인***(Domain)***을 이용해서 소프트웨어***(SW)***를 생성하고 디버깅 ***(Dubugging)***을 진행
  - ***Figure: Embedded Software Application Development Workflow***



  - ***VITIS***를 이용한 전체 소프트웨어 구현 프로세스



수고하셨습니다.